

第 12 章

图像轮廓

边缘检测虽然能够检测出边缘，但边缘是不连续的，检测到的边缘并不是一个整体。图像轮廓是指将边缘连接起来形成的一个整体，用于后续的计算。

OpenCV 提供了查找图像轮廓的函数 `cv2.findContours()`，该函数能够查找图像内的轮廓信息，而函数 `cv2.drawContours()` 能够将轮廓绘制出来。

图像轮廓是图像中非常重要的一个特征信息，通过对图像轮廓的操作，我们能够获取目标图像的大小、位置、方向等信息。

12.1 查找并绘制轮廓

一个轮廓对应着一系列的点，这些点以某种方式表示图像中的一条曲线。在 OpenCV 中，函数 `cv2.findContours()` 用于查找图像的轮廓，并能够根据参数返回特定表示方式的轮廓(曲线)。函数 `cv2.drawContours()` 能够将查找到的轮廓绘制到图像上，该函数可以根据参数在图像上绘制不同样式(实心/空心点，以及线条的不同粗细、颜色等)的轮廓，可以绘制全部轮廓也可以仅绘制指定的轮廓。

12.1.1 查找图像轮廓：findContours函数

函数 `cv2.findContours()` 的语法格式为：

```
image, contours, hierarchy = cv2.findContours( image, mode, method)
```

式中的返回值为：

- **image**：与函数参数中的原始图像 `image` 一致。
- **contours**：返回的轮廓。
- **hierarchy**：图像的拓扑信息（轮廓层次）。

式中的参数为：

- **image**：原始图像。8 位单通道图像，所有非零值被处理为 1，所有零值保持不变。也就是说灰度图像会被自动处理为二值图像。在实际操作时，可以根据需要，预先使用阈值处理等函数将待查找轮廓的图像处理为二值图像。
- **mode**：轮廓检索模式。
- **method**：轮廓的近似方法。



函数 `cv2.findContours()` 的返回值及参数的含义比较丰富，下面对上述返回值和参数逐一做出说明。

1. 返回值 `image`

该返回值与参数 `image` 是一致的，就是原始输入图像。在 OpenCV 4.X 中，该返回值已经被取消。在 OpenCV 4.X 中，函数 `cv2.findContours()` 仅有两个返回值，其语法格式为：

```
contours, hierarchy = cv2.findContours( image, mode, method)
```

2. 返回值 `contours`

该返回值返回的是一组轮廓信息，每个轮廓都是由若干个点所构成的。例如，`contours[i]` 是第 i 个轮廓（下标从 0 开始），`contours[i][j]` 是第 i 个轮廓内的第 j 个点。

图 12-1 所示为提取的轮廓示例，函数 `cv2.findContours()` 提取出左图的 3 个轮廓，每一个轮廓都是由若干个像素点构成的。

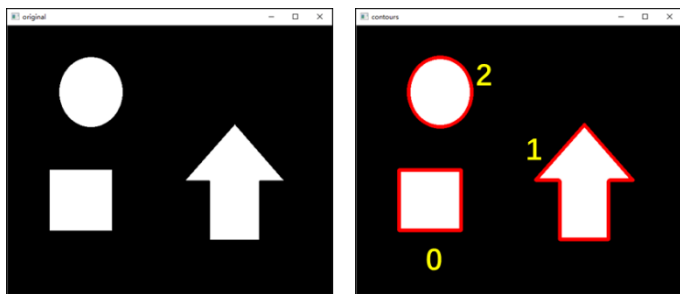


图 12-1 轮廓示例

下面针对图 12-1 来简单介绍一下 `contours` 的基本属性。

(1) `type` 属性

返回值 `contours` 的 `type` 属性是 `list` 类型，`list` 的每个元素都是图像的一个轮廓，用 Numpy 中的 `ndarray` 结构表示。

例如，使用如下语句获取轮廓 `contours` 的类型：

```
print (type(contours))
```

结果为 `<class 'list'>`。

使用如下语句获取轮廓 `contours` 中每个元素的类型：

```
print (type(contours[0]))
```

结果为 `<class 'numpy.ndarray'>`。

(2) 轮廓的个数

使用如下语句可以获取轮廓的个数：

```
print (len(contours))
```

结果为“3”，表示在图 12-1 中，存在 3 个轮廓。

(3) 每个轮廓的点数

每一个轮廓都是由若干个像素点构成的，点的个数不固定，具体个数取决于轮廓的形状。

例如，使用如下语句，可以获取每个轮廓内点的个数：

```
print (len(contours[0]))    #打印第 0 个轮廓的长度（点的个数）： 4
print (len(contours[1]))    #打印第 1 个轮廓的长度（点的个数）： 60
print (len(contours[2]))    #打印第 2 个轮廓的长度（点的个数）： 184
```

输出结果如下：

```
4
60
184
```

使用如下语句，可以获取每个轮廓内点的 shape 属性：

```
print(contours[0].shape)
print(contours[1].shape)
print(contours[2].shape)
```

输出结果如下：

```
(4, 1, 2)
(60, 1, 2)
(184, 1, 2)
(4) 轮廓内的点
```

使用如下语句，可以获取轮廓内第 0 个轮廓中具体点的位置属性：

```
print (contours[0])    #打印第 0 个轮廓中的像素点
```

contours[0]对应着图 12-1 中右图左下角矩形轮廓的点，输出结果如下：

```
[[[ 79 270]]]
[[ 79 383]]
[[195 383]]
[[195 270]]]
```

3. 返回值hierarchy

图像内的轮廓可能位于不同的位置。比如，一个轮廓在另一个轮廓的内部。在这种情况下，我们将外部的轮廓称为父轮廓，内部的轮廓称为子轮廓。按照上述关系分类，一幅图像中所有轮廓之间就建立了父子关系。

根据轮廓之间的关系，就能够确定一个轮廓与其他轮廓是如何连接的。比如，确定一个轮廓是某个轮廓的子轮廓，或者是某个轮廓的父轮廓。上述关系被称为层次（组织结构），返回值 hierarchy 就包含上述层次关系。

每个轮廓 contours[i]对应 4 个元素来说明当前轮廓的层次关系。其形式为：

```
[Next, Previous, First_Child, Parent]
```

式中各元素的含义为：

- Next: 后一个轮廓的索引编号。
- Previous: 前一个轮廓的索引编号。
- First_Child: 第 1 个子轮廓的索引编号。



- Parent: 父轮廓的索引编号。

如果上述各个参数所对应的关系为空时，也就是没有对应的关系时，则将该参数所对应的值设为“-1”。

使用 print 语句可以查看 hierarchy 的值：

```
print(hierarchy)
```

需要注意，轮廓的层次结构是由参数 mode 决定的。也就是说，使用不同的 mode，得到轮廓的编号是不一样的，得到的 hierarchy 也不一样。

4. 参数image

该参数表示输入的图像，必须是 8 位单通道二值图像。一般情况下，都是将图像处理为二值图像后，再将其作为 image 参数使用的。

5. 参数mode

参数 mode 决定了轮廓的提取方式，具体有如下 4 种：

- cv2.RETR_EXTERNAL: 只检测外轮廓。
- cv2.RETR_LIST: 对检测到的轮廓不建立等级关系。
- cv2.RETR_CCOMP: 检索所有轮廓并将它们组织成两级层次结构。上面的一层为外边界，下面的一层为内孔的边界。如果内孔内还有一个连通物体，那么这个物体的边界仍然位于顶层。
- cv2.RETR_TREE: 建立一个等级树结构的轮廓。

下面分别对这四种情况进行简单的说明。

(1) cv2.RETR_EXTERNAL (只检测外轮廓)

例如，在图 12-2 中仅检测到两个外轮廓，轮廓的序号如图中的数字标注所示。

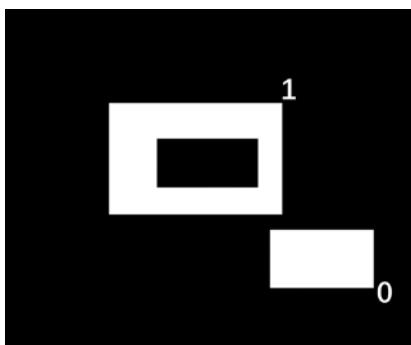


图 12-2 轮廓示意图 (1)

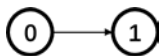
使用 print 语句可以查看 hierarchy 的值：

```
print(hierarchy)
[[[ 1 -1 -1 -1]
  [-1 0 -1 -1]]]
```

其中:

- 输出值 “[1 -1 -1 -1]”，表示的是第 0 个轮廓的层次。
 - 它（即第 0 个轮廓）的后一个轮廓就是第 1 个轮廓，因此第 1 个元素的值是 “1”。
 - 它的前一个轮廓不存在，因此第 2 个元素的值是 “-1”。
 - 它不存在子轮廓，因此第 3 个元素的值是 “-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是 “-1”。
- 输出值 “[-1 0 -1 -1]”，表示的是第 1 个轮廓的层次。
 - 它（即第 1 个轮廓）的后一个轮廓是不存在的，因此第 1 个元素的值是 “-1”。
 - 它的前一个轮廓是第 0 个轮廓，因此第 2 个元素的值是 “0”。
 - 它不存在子轮廓，因此第 3 个元素的值是 “-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是 “-1”。

此时，轮廓之间的关系为:



(2) cv2.RETR_LIST（对检测到的轮廓不建立等级关系）

例如，在图 12-3 中检测到三个轮廓，各个轮廓的序号如图中数字的标注所示。

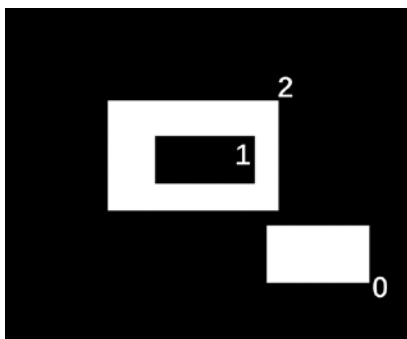


图 12-3 轮廓示意图 (2)

使用 print 语句，可以查看 hierarchy 的值:

```
print(hierarchy)
[[[ 1 -1 -1 -1]
  [ 2  0 -1 -1]
  [-1  1 -1 -1]]]
```

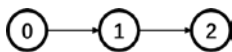
其中:

- 输出值 “[1 -1 -1 -1]”，表示的是第 0 个轮廓的层次。
 - 它（即第 0 个轮廓）的后一个轮廓是第 1 个轮廓，因此第 1 个元素的值是 “1”。
 - 它的前一个轮廓不存在，因此第 2 个元素的值是 “-1”。
 - 它不存在子轮廓，因此第 3 个元素的值是 “-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是 “-1”。
- 输出值 “[2 0 -1 -1]”，表示的是第 1 个轮廓的层次。

- 它（即第 1 个轮廓）的后一个轮廓是第 2 个轮廓，因此第 1 个元素的值是“2”。
- 它的前一个轮廓是第 0 个轮廓，因此第 2 个元素的值是“0”。
- 它不存在子轮廓，因此第 3 个元素的值是“-1”。
- 它不存在父轮廓，因此第 4 个元素的值是“-1”。
- 输出值“[-1 1 -1 -1]”，表示的是第 2 个轮廓的层次。
 - 它（即第 2 个轮廓）的后一个轮廓是不存在的，因此第 1 个元素的值是“-1”。
 - 它的前一个轮廓是第 1 个轮廓，因此第 2 个元素的值是“1”。
 - 它不存在子轮廓，因此第 3 个元素的值是“-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是“-1”。

从上述分析可以看出，当参数 `mode` 为 `cv2.RETR_LIST` 时，各个轮廓之间没有建立父子关系。

此时，轮廓之间的关系为：



(3) `cv2.RETR_CCOMP`（建立两个等级的轮廓）

当参数 `mode` 为 `cv2.RETR_CCOMP` 时，建立两个等级的轮廓。上面的一层为外边界，下面的一层为内孔边界。

例如，在图 12-4 中检测到三个轮廓，各轮廓的序号如图中数字的标注所示。

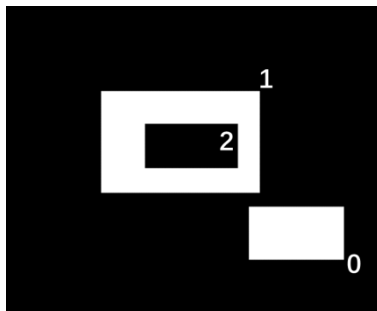


图 12-4 轮廓示意图（3）

使用 `print` 语句可以查看 `hierarchy` 的值：

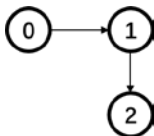
```
print(hierarchy)
[[[ 1 -1 -1 -1]
  [-1 0 2 -1]
  [-1 -1 -1 1]]]
```

其中：

- 输出值“[1 -1 -1 -1]”，表示的是第 0 个轮廓的层次。
 - 它（即第 0 个轮廓）的后一个轮廓是第 1 个轮廓，因此第 1 个元素的值是“1”。
 - 它的前一个轮廓不存在，因此第 2 个元素的值是“-1”。
 - 它不存在子轮廓，因此第 3 个元素的值是“-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是“-1”。

- 输出值 “[-1 0 2 -1]”，表示的是第 1 个轮廓的层次。
 - 它（即第 1 个轮廓）的后一个轮廓不存在，因此第 1 个元素的值是“-1”。
 - 它的前一个轮廓是第 0 个轮廓，因此第 2 个元素的值是“0”。
 - 它的第 1 个子轮廓是第 2 个轮廓，因此第 3 个元素的值是“2”。
 - 它不存在父轮廓，因此第 4 个元素的值是“-1”。
- 输出值 “[-1 -1 -1 1]”，表示的是第 2 个轮廓的层次。
 - 它（即第 2 个轮廓）的后一个轮廓不存在，因此第 1 个元素的值是“-1”。
 - 它的前一个轮廓也不存在，因此第 2 个元素的值是“-1”。
 - 它不存在子轮廓，因此第 3 个元素的值是“-1”。
 - 它的父轮廓是第 1 个轮廓，因此第 4 个元素的值是“1”。

此时，轮廓关系为：



(4) cv2.RETR_TREE（建立一个等级树结构的轮廓）

例如，在图 12-5 中检测到三个轮廓，各个轮廓的序号如图中的数字标注所示。

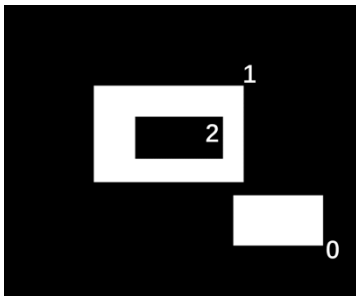


图 12-5 轮廓示意图（4）

使用 print 语句可以查看 hierarchy 的值：

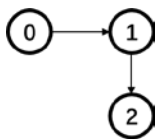
```
print(hierarchy)
[[[ 1 -1 -1 -1]
  [-1 0 2 -1]
  [-1 -1 -1 1]]]
```

其中：

- 输出值 “[1 -1 -1 -1]”，表示的是第 0 个轮廓的层次。
 - 它（即第 0 个轮廓）的后一个轮廓是第 1 个轮廓，因此第 1 个元素的值为“1”。
 - 它的前一个轮廓不存在，因此第 2 个元素的值是“-1”。
 - 它不存在子轮廓，因此第 3 个元素的值是“-1”。
 - 它不存在父轮廓，因此第 4 个元素的值是“-1”。
- 输出值 “[-1 0 2 -1]”，表示的是第 1 个轮廓的层次。

- 它（即第 1 个轮廓）的后一个轮廓不存在，因此第 1 个元素的值是“-1”。
- 它的前一个轮廓是第 0 个轮廓，因此第 2 个元素的值是“0”。
- 它的第 1 个子轮廓是第 2 个轮廓，因此第 3 个元素的值是“2”。
- 它的父轮廓不存在，因此第 4 个元素的值是“-1”。
- 输出值“[-1 -1 -1 1]”，表示的是第 2 个轮廓的层次。
 - 它（即第 2 个轮廓）的后一个轮廓不存在，因此第 1 个元素的值是“-1”。
 - 它的前一个轮廓是不存在的，因此第 2 个元素的值是“-1”。
 - 它的子轮廓是不存在的，因此第 3 个元素的值是“-1”。
 - 它的父轮廓是第 1 个轮廓，因此第 4 个元素的值是“1”。

此时，轮廓之间的关系为：



需要注意，本例中仅有两层轮廓，所以使用参数 `cv2.RETR_CCOMP` 和 `cv2.RETR_TREE` 得到的层次结构是一致的。当有多层轮廓时，使用参数 `cv2.RETR_CCOMP` 也会得到仅有两层的层次结构；而使用参数 `cv2.RETR_TREE` 会得到含有多个层次的结构。限于篇幅，这里不再列举更多层轮廓的层次关系。

6. 参数method

参数 `method` 决定了如何表达轮廓，可以为如下值：

- `cv2.CHAIN_APPROX_NONE`：存储所有的轮廓点，相邻两个点的像素位置差不超过 1，即 $\max(\text{abs}(x_1 - x_2), \text{abs}(y_2 - y_1)) = 1$ 。
- `cv2.CHAIN_APPROX_SIMPLE`：压缩水平方向、垂直方向、对角线方向的元素，只保留该方向的终点坐标。例如，在极端的情况下，一个矩形只需要用 4 个点来保存轮廓信息。
- `cv2.CHAIN_APPROX_TC89_L1`：使用 teh-Chinl chain 近似算法的一种风格。
- `cv2.CHAIN_APPROX_TC89_KCOS`：使用 teh-Chinl chain 近似算法的一种风格。

例如，在图 12-6 中，左图是使用参数值 `cv2.CHAIN_APPROX_NONE` 存储的轮廓，保存了轮廓中的每一个点；右图是使用参数值 `cv2.CHAIN_APPROX_SIMPLE` 存储的轮廓，仅仅保存了边界上的四个点。

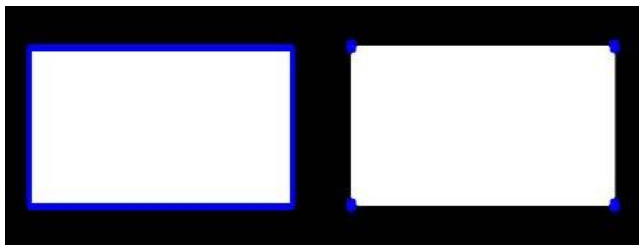


图 12-6 使用不同的 `method` 参数值所保存的轮廓点

在使用函数 `cv2.findContours()` 查找图像轮廓时，需要注意以下问题：

- 待处理的源图像必须是灰度二值图。因此，在通常情况下，都要预先对图像进行阈值分割或者边缘检测处理，得到满意的二值图像后再将其作为参数使用。
- 在 OpenCV 中，都是从黑色背景中查找白色对象。因此，对象必须是白色的，背景必须是黑色的。
- 在 OpenCV 4.x 中，函数 `cv2.findContours()` 仅有两个返回值。

12.1.2 绘制图像轮廓：drawContours 函数

在 OpenCV 中，可以使用函数 `cv2.drawContours()` 绘制图像轮廓。该函数的语法格式是：

```
image=cv2.drawContours(
image,
contours,
contourIdx,
color[,
thickness[,
lineType[,
hierarchy[,
maxLevel[,
offset]]]] )
```

其中，函数的返回值为 `image`，表示目标图像，即绘制了边缘的原始图像。

该函数有如下参数：

- **image**：待绘制轮廓的图像。需要注意，函数 `cv2.drawContours()` 会在图像 `image` 上直接绘制轮廓。也就是说，在函数执行完以后，`image` 不再是原始图像，而是包含了轮廓的图像。因此，如果图像 `image` 还有其他用途的话，则需要预先复制一份，将该副本图像传递给函数 `cv2.drawContours()` 使用。
- **contours**：需要绘制的轮廓。该参数的类型与函数 `cv2.findContours()` 的输出 `contours` 相同，都是 `list` 类型。
- **contourIdx**：需要绘制的边缘索引，告诉函数 `cv2.drawContours()` 要绘制某一条轮廓还是全部轮廓。如果该参数是一个整数或者为零，则表示绘制对应索引号的轮廓；如果该值为负数（通常为“-1”），则表示绘制全部轮廓。
- **color**：绘制的颜色，用 BGR 格式表示。
- **thickness**：可选参数，表示绘制轮廓时所用画笔的粗细。如将该值设置为“-1”，则表示要绘制实心轮廓。
- **lineType**：可选参数，表示绘制轮廓时所用的线型。关于参数 `color`、`thickness` 和 `lineType` 的具体介绍，可参考第 19 章。
- **hierarchy**：对应函数 `cv2.findContours()` 所输出的层次信息。
- **maxLevel**：控制所绘制的轮廓层次的深度。如果值为 0，表示仅仅绘制第 0 层的轮廓；如果值为其他的非零正数，表示绘制最高层及以下的相同数量层级的轮廓。



- **offset**: 偏移参数。该参数使轮廓偏移到的位置展示出来。

函数 `cv2.drawContours()` 的参数 `image` 和返回值 `image`, 在函数运算后的值是相同的。因此, 也可以将函数 `cv2.drawContours()` 写为没有返回值的形式:

```
cv2.drawContours(
    image,
    contours,
    contourIdx,
    color[,
    thickness[,
    lineType[,
    hierarchy[,
    maxLevel[,
    offset]]]] )
```

12.1.3 轮廓实例

本节通过具体实例说明如何获取轮廓、绘制轮廓。

【例 12.1】 绘制一幅图像内的所有轮廓。

如果要绘制图像内的所有轮廓, 需要将函数 `cv2.drawContours()` 的参数 `contourIdx` 的值设置为 “-1”。

根据题目的要求及分析, 编写代码如下:

```
import cv2
o = cv2.imread('contours.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
                                             cv2.RETR_EXTERNAL,
                                             cv2.CHAIN_APPROX_SIMPLE)
o=cv2.drawContours(o,contours, -1, (0,0,255),5)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本程序中, 轮廓的颜色被设置为红色 (由于黑白印刷的原因, 在纸质书中显示为灰色), 即 (0, 0, 255), 参数 `thickness` (轮廓线条的粗细) 被设置为 “5”。

运行上述程序, 结果如图 12-7 所示, 图像内的所有轮廓都被绘制出来了。

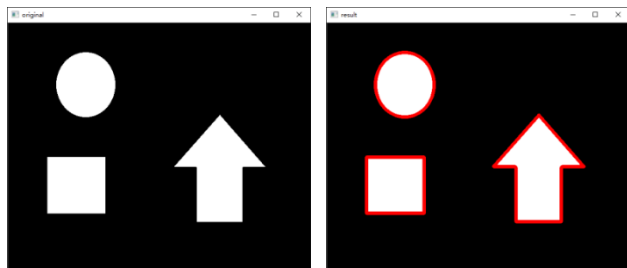


图 12-7 绘制所有轮廓

【例 12.2】 逐个显示一幅图像内的边缘信息。

如果要绘制图像内的某个具体轮廓，需要将函数 `cv2.drawContours()` 的参数 `contourIdx` 设置为具体的索引值。本例通过循环语句逐一绘制轮廓。

根据题目要求及分析，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('contours.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                           cv2.RETR_EXTERNAL,
                                           cv2.CHAIN_APPROX_SIMPLE)

n=len(contours)
contoursImg=[]
for i in range(n):
    temp=np.zeros(o.shape,np.uint8)
    contoursImg.append(temp)
    contoursImg[i]=cv2.drawContours(
        contoursImg[i],contours,i,(255,255,255),5)
    cv2.imshow("contours["+ str(i)+"]",contoursImg[i])
cv2.waitKey()
cv2.destroyAllWindows()
```

为了方便读者在印刷的纸质书上能够清晰地观察到轮廓，本程序中将轮廓设置为白色，即 (255, 255, 255)。

运行上述程序，结果如图 12-8 所示，图像内的轮廓被逐一绘制出来。



图 12-8 逐一显示所有轮廓



【例 12.3】使用轮廓绘制功能，提取前景对象。

将函数 `cv2.drawContours()` 的参数 `thickness` 的值设置为“-1”，可以绘制前景对象的实心轮廓。将该实心轮廓与原始图像进行“按位与”操作，即可将前景对象从原始图像中提取出来。

根据题目的要求及分析，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('loc3.jpg')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                     cv2.RETR_LIST,
                                     cv2.CHAIN_APPROX_SIMPLE)

mask=np.zeros(o.shape,np.uint8)
mask=cv2.drawContours(mask,contours,-1,(255,255,255),-1)
cv2.imshow("mask",mask)
loc=cv2.bitwise_and(o,mask)
cv2.imshow("location",loc)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例中将函数 `cv2.drawContours()` 的参数 `thickness` 设置为“-1”，得到了前景对象的实心轮廓 `mask`。接下来，通过语句“`cv2.bitwise_and(o, mask)`”，将原始图像 `o` 与实心轮廓 `mask` 进行“按位与”运算，就得到了原始图像的前景对象。

运行上述程序，结果如图 12-9 所示。其中：

- 左图是原始图像，其前景对象是一朵小花。
- 中间的图像是从原始图像得到的小花的实心轮廓。
- 右图是提取的前景对象小花。

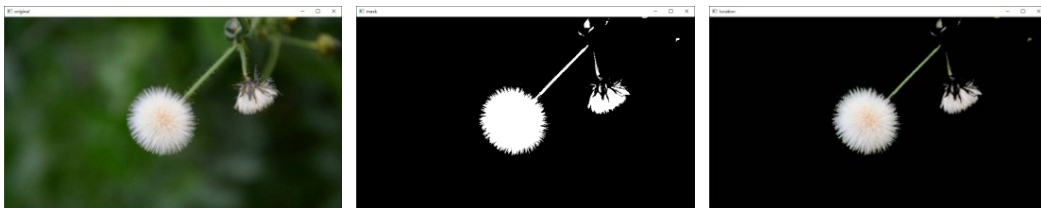


图 12-9 显示前景对象的轮廓及前景对象

12.2 矩特征

比较两个轮廓最简单的方法是比较二者的轮廓矩。轮廓矩代表了一个轮廓、一幅图像、一组点集的全局特征。矩信息包含了对应对象不同类型的几何特征，例如大小、位置、角度、形状等。矩特征被广泛地应用在模式识别、图像识别等方面。

12.2.1 矩的计算：moments函数

OpenCV 提供了函数 `cv2.moments()` 来获取图像的 moments 特征。通常情况下，我们将使用函数 `cv2.moments()` 获取的轮廓特征称为“轮廓矩”。轮廓矩描述了一个轮廓的重要特征，使用轮廓矩可以方便地比较两个轮廓。

函数 `cv2.moments()` 的语法格式为：

```
retval = cv2.moments( array[, binaryImage] )
```

式中有两个参数：

- **array**：可以是点集，也可以是灰度图像或者二值图像。当 **array** 是点集时，函数会把点集当成轮廓中的顶点，把整个点集作为一条轮廓，而不是把它们当成独立的点来看待。
- **binaryImage**：该参数为 `True` 时，**array** 内所有的非零值都被处理为 1。该参数仅在参数 **array** 为图像时有效。

该函数的返回值 **retval** 是矩特征，主要包括：

(1) 空间矩

- 零阶矩：m00
- 一阶矩：m10, m01
- 二阶矩：m20, m11, m02
- 三阶矩：m30, m21, m12, m03

(2) 中心矩

- 二阶中心矩：mu20, mu11, mu02
- 三阶中心矩：mu30, mu21, mu12, mu03

(3) 归一化中心矩

- 二阶 Hu 矩：nu20, nu11, nu02
- 三阶 Hu 矩：nu30, nu21, nu12, nu03

上述矩都是根据公式计算得到的，大多数矩比较抽象。但是很明显，如果两个轮廓的矩一致，那么这两个轮廓就是一致的。虽然大多数矩都是通过数学公式计算得到的抽象特征，但是零阶矩“m00”的含义比较直观，它表示一个轮廓的面积。

矩特征函数 `cv2.moments()` 所返回的特征值，能够用来比较两个轮廓是否相似。例如，有两个轮廓，不管它们出现在图像的哪个位置，我们都可以通过函数 `cv2.moments()` 的 m00 矩判断其面积是否一致。

在位置发生变化时，虽然轮廓的面积、周长等特征不变，但是更高阶的特征会随着位置的变化而发生变化。在很多情况下，我们希望比较不同位置的两个对象的一致性。解决这一问题的方法是引入中心矩。中心矩通过减去均值而获取平移不变性，因而能够比较不同位置的两个对象是否一致。很明显，中心矩具有的平移不变性，使它能够忽略两个对象的位置关系，帮助我们比较不同位置上两个对象的一致性。



除了考虑平移不变性外，我们还会考虑经过缩放后大小不一致的对象的一致性。也就是说，我们希望图像在缩放前后能够拥有一个稳定的特征值。也就是说，让图像在缩放前后具有同样的特征值。显然，中心矩不具有这个属性。例如，两个形状一致、大小不一的对象，其中心矩是有差异的。

归一化中心矩通过除以物体总尺寸而获得缩放不变性。它通过上述计算提取对象的归一化中心矩属性值，该属性值不仅具有平移不变性，还具有缩放不变性。

在 OpenCV 中，函数 `cv2.moments()` 会同时计算上述空间矩、中心矩和归一化中心矩。

【例 12.4】 使用函数 `cv2.moments()` 提取一幅图像的特征。

根据题目的要求及分析，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('moments.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)

n=len(contours)
contoursImg=[]
for i in range(n):
    temp=np.zeros(image.shape,np.uint8)
    contoursImg.append(temp)
    contoursImg[i]=cv2.drawContours(contoursImg[i],contours,i,255,3)
    cv2.imshow("contours["+str(i)+"]",contoursImg[i])
print("观察各个轮廓的矩（moments）：")
for i in range(n):
    print("轮廓"+str(i)+"的矩:\n",cv2.moments(contours[i]))
print("观察各个轮廓的面积:")
for i in range(n):
    print("轮廓"+str(i)+"的面积:%d" %cv2.moments(contours[i])['m00'])
cv2.waitKey()
cv2.destroyAllWindows()
```

本例中，首先使用函数 `cv2.moments()` 提取各个轮廓的特征；接下来，通过语句 `cv2.moments(contours[i])['m00']` 提取各个轮廓矩的面积信息。

运行上述程序，会显示如图 12-10 所示的图像。其中：

- (a)图是原始图像。
- (b)图是原始图像的第 0 个轮廓。
- (c)图是原始图像的第 1 个轮廓。
- (d)图是原始图像的第 2 个轮廓。

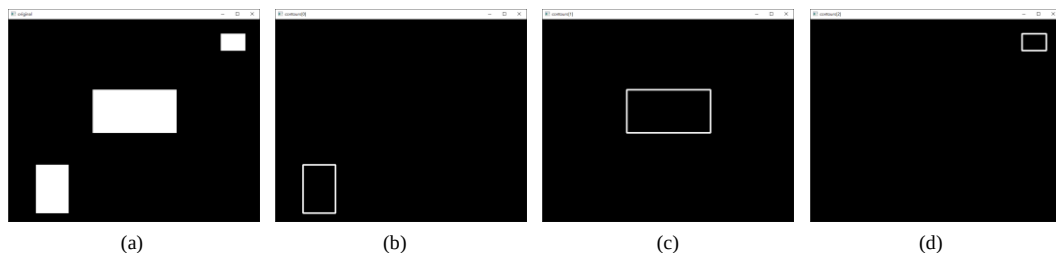


图 12-10 显示所有轮廓

同时，程序会显示如下输出结果：

观察各个轮廓的矩 (moments)：

轮廓 0 的矩：

```
{'m00': 14900.0, 'm10': 1996600.0, 'm01': 7800150.0, 'm20': 279961066.6666666,
'm11': 1045220100.0, 'm02': 4110944766.6666665, 'm30': 40842449600.0, 'm21':
146559618400.0, 'm12': 550866598733.3334, 'm03': 2180941440375.0, 'mu20':
12416666.666666627, 'mu11': 0.0, 'mu02': 27566241.666666508, 'mu30':
1.52587890625e-05, 'mu21': 2.09808349609375e-05, 'mu12': 6.198883056640625e-05,
'mu03': 0.000244140625, 'nu20': 0.05592841163310942, 'nu11': 0.0, 'nu02':
0.12416666666666591, 'nu30': 5.630596400372416e-16, 'nu21': 7.742070050512072e-16,
'nu12': 2.2874297876512943e-15, 'nu03': 9.008954240595866e-15}
```

轮廓 1 的矩：

```
{'m00': 34314.0, 'm10': 13313832.0, 'm01': 9728019.0, 'm20': 5356106574.0, 'm11':
3774471372.0, 'm02': 2808475082.0, 'm30': 2225873002920.0, 'm21': 1518456213729.0,
'm12': 1089688331816.0, 'm03': 824882507095.5, 'mu20': 190339758.0, 'mu11': 0.0,
'mu02': 50581695.5, 'mu30': 0.0, 'mu21': 0.0, 'mu12': 0.0, 'mu03': 0.0, 'nu20':
0.16165413533834588, 'nu11': 0.0, 'nu02': 0.042958656330749356, 'nu30': 0.0, 'nu21':
0.0, 'nu12': 0.0, 'nu03': 0.0}
```

轮廓 2 的矩：

```
{'m00': 3900.0, 'm10': 2696850.0, 'm01': 273000.0, 'm20': 1866699900.0, 'm11':
188779500.0, 'm02': 19988800.0, 'm30': 1293351277725.0, 'm21': 130668993000.0, 'm12':
13822255200.0, 'm03': 1522248000.0, 'mu20': 1828125.0, 'mu11': 0.0, 'mu02': 878800.0,
'mu30': 0.0, 'mu21': 0.0, 'mu12': 0.0, 'mu03': 0.0, 'nu20': 0.1201923076923077,
'nu11': 0.0, 'nu02': 0.05777777777777778, 'nu30': 0.0, 'nu21': 0.0, 'nu12': 0.0,
'nu03': 0.0}
```

观察各个轮廓的面积：

轮廓 0 的面积:14900

轮廓 1 的面积:34314

轮廓 2 的面积:3900

上述输出结果显示了每个轮廓的矩特征和面积。

12.2.2 计算轮廓的面积：contourArea函数

函数 `cv2.contourArea()` 用于计算轮廓的面积。该函数的语法格式为：

```
retval =cv2.contourArea(contour [, oriented] ))
```

式中的返回值 `retval` 是面积值。



式中有两个参数：

- `contour` 是轮廓。
- `oriented` 是布尔型值。当它为 `True` 时，返回的值包含正/负号，用来表示轮廓是顺时针还是逆时针的。该参数的默认值是 `False`，表示返回的 `retval` 是一个绝对值。

【例 12.5】 使用函数 `cv2.contourArea()` 计算各个轮廓的面积。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('contours.bmp')
gray = cv2.cvtColor(o, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cv2.imshow("original", o)
n = len(contours)
contoursImg = []
for i in range(n):
    print("contours[" + str(i) + "]面积=", cv2.contourArea(contours[i]))
    temp = np.zeros(o.shape, np.uint8)
    contoursImg.append(temp)
    contoursImg[i] = cv2.drawContours(contoursImg[i],
                                     contours,
                                     i,
                                     (255, 255, 255),
                                     3)
    cv2.imshow("contours[" + str(i) + "]", contoursImg[i])
cv2.waitKey()
cv2.destroyAllWindows()
```

本例通过函数 `cv2.contourArea()` 计算各个轮廓的面积。运行上述程序，会显示各个轮廓的面积值：

```
contours[0]面积= 13108.0
contours[1]面积= 19535.0
contours[2]面积= 12058.0
```

同时，还会显示如图 12-11 所示的图像。其中：

- (a)图是原始图像。
- (b)图是原始图像的第 0 个轮廓。
- (c)图是原始图像的第 1 个轮廓。
- (d)图是原始图像的第 2 个轮廓。



图 12-11 显示所有轮廓

【例 12.6】 在例 12.5 的基础上，将面积大于 15 000 的轮廓筛选出来。

我们在例 12.5 中已经计算出图像 contours.bmp 内三个轮廓的面积分别是 13108.0、19535.0、12058.0，使用 if 函数将面积大于 15 000 的值筛选出来即可。

根据题目的要求及分析，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('contours.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

n=len(contours)
contoursImg=[]
for i in range(n):
    temp=np.zeros(o.shape,np.uint8)
    contoursImg.append(temp)
    contoursImg[i]=cv2.drawContours(contoursImg[i],
                                   contours,i,(255,255,255),3)
    if cv2.contourArea(contours[i])>15000:
        cv2.imshow("contours["+ str(i)+"]",contoursImg[i])
cv2.waitKey()
cv2.destroyAllWindows()
```

本例中，通过语句“if cv2.contourArea(contours[i])>15000:”实现对面积值的筛选，然后对面积值大于 15 000 的轮廓使用语句“cv2.imshow("contours["+ str(i)+"]",contoursImg[i])”显示出来。

运行上述程序，显示的图像如图 12-11 所示。其中：

- 左图是原始图像。
- 右图是原始图像内面积大于 15 000 的唯一轮廓。

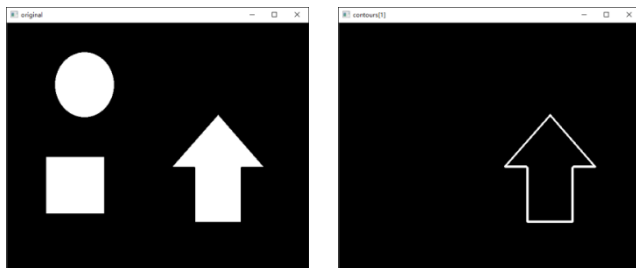


图 12-12 显示面积大于 15 000 的轮廓

12.2.3 计算轮廓的长度：arcLength函数

函数 `cv2.arcLength()` 用于计算轮廓的长度，其语法格式为：

```
retval = cv2.arcLength( curve, closed )
```

式中返回值 `retval` 是轮廓的长度（周长）。

上式中有两个参数：

- `curve` 是轮廓。
- `closed` 是布尔型值，用来表示轮廓是否是封闭的。该值为 `True` 时，表示轮廓是封闭的。

【例 12.7】 将一幅图像内长度大于平均值的轮廓显示出来。

首先，使用函数 `cv2.arcLength()` 计算各个轮廓的长度，接下来计算各个轮廓的长度之和以及长度的平均值，最后使用判断语句将长度大于平均值的轮廓显示出来。

根据题目的要求及分析，编写代码如下：

```
import cv2
import numpy as np
#-----读取及显示原始图像-----
o = cv2.imread('contours0.bmp')
cv2.imshow("original",o)
#-----获取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
                                             cv2.RETR_LIST,
                                             cv2.CHAIN_APPROX_SIMPLE)
#-----计算各轮廓的长度之和、平均长度-----
n=len(contours) # 获取轮廓的个数
cntLen=[]       # 存储各轮廓的长度
for i in range(n):
    cntLen.append(cv2.arcLength(contours[i],True))
    print("第"+str(i)+"个轮廓的长度:%d"%cntLen[i])
cntLenSum=np.sum(cntLen) # 各轮廓的长度之和
cntLenAvr=cntLenSum/n   # 轮廓长度的平均值
print("轮廓的总长度为: %d"%cntLenSum)
print("轮廓的平均长度为: %d"%cntLenAvr)
```

```
#-----显示长度超过平均值的轮廓-----
contoursImg=[]
for i in range(n):
    temp=np.zeros(o.shape,np.uint8)
    contoursImg.append(temp)
    contoursImg[i]=cv2.drawContours(contoursImg[i],
                                    contours,i,(255,255,255),3)
    if cv2.arcLength(contours[i],True)>cntLenAvr:
        cv2.imshow("contours["+str(i)+"]",contoursImg[i])
cv2.waitKey()
cv2.destroyAllWindows()
```

本例先通过函数 `cv2.arcLength()` 计算各个轮廓的长度，并据此计算总长度和平均长度，然后使用判断结构语句 “`if cv2.arcLength(contours[i],True)>cntLenAvr:`” 对各个轮廓的长度进行判断，并根据判断结果决定是否显示对应的轮廓。

运行上述程序，会显示各个轮廓的长度、总长度与平均长度。

第 0 个轮廓的长度:145
 第 1 个轮廓的长度:147
 第 2 个轮廓的长度:398
 第 3 个轮廓的长度:681
 第 4 个轮廓的长度:1004
 第 5 个轮廓的长度:398
 第 6 个轮廓的长度:681
 第 7 个轮廓的长度:1004
 第 8 个轮廓的长度:2225
 第 9 个轮廓的长度:2794
 轮廓的总长度为: 9480
 轮廓的平均长度为: 948

同时，程序还会显示如图 12-13 所示的图像，其中显示了原始图像和长度大于平均轮廓长度的 4 条轮廓：

- (a)图是原始图像。
- (b)图是原始图像的第 4 个轮廓。
- (c)图是原始图像的第 7 个轮廓。
- (d)图是原始图像的第 8 个轮廓。
- (e)图是原始图像的第 9 个轮廓（最外层，即整体图像的外轮廓）。

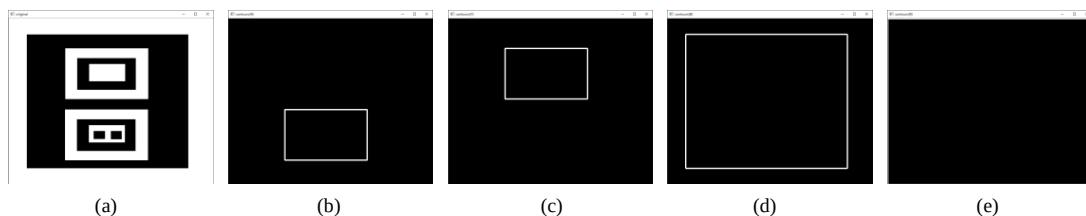


图 12-13 显示长度大于平均长度的轮廓

12.3 Hu 矩

Hu 矩是归一化中心矩的线性组合。Hu 矩在图像旋转、缩放、平移等操作后，仍能保持矩的不变性，所以经常会使用 Hu 矩来识别图像的特征。

在 OpenCV 中，使用函数 `cv2.HuMoments()` 可以得到 Hu 矩。该函数使用 `cv2.moments()` 函数的返回值作为参数，返回 7 个 Hu 矩值。

12.3.1 Hu 矩函数

函数 `cv2.HuMoments()` 的语法格式为：

```
hu = cv2.HuMoments( m )
```

式中返回值 `hu`，表示返回的 Hu 矩值；参数 `m`，是由函数 `cv2.moments()` 计算得到矩特征值。

【例 12.8】 计算图像的 Hu 矩，对其中第 0 个矩的关系进行演示。

Hu 矩是归一化中心矩的线性组合，每一个矩都是通过归一化中心矩的组合运算得到的。函数 `cv2.moments()` 返回的归一化中心矩中包含：

- 二阶 Hu 矩：nu20, nu11, nu02
- 三阶 Hu 矩：nu30, nu21, nu12, nu03

为了表述上的方便，将上述字母“nu”表示为字母“v”，则归一化中心矩为：

- 二阶 Hu 矩：v20, v11, v02
- 三阶 Hu 矩：v30, v21, v12, v03

上述 7 个 Hu 矩的计算公式为：

$$\begin{aligned}
 h_0 &= v_{20} + v_{02} \\
 h_1 &= (v_{20} - v_{02})^2 + 4v_{11}^2 \\
 h_2 &= (v_{30} - 3v_{12})^2 + (3v_{21} - v_{03})^2 \\
 h_3 &= (v_{30} + v_{12})^2 + (v_{21} + v_{03})^2 \\
 h_4 &= (v_{30} - 3v_{12})(v_{30} + v_{12}) \left[(v_{30} - 3v_{12})^2 - 3(v_{21} + v_{03})^2 \right] \\
 &\quad + (3v_{21} - v_{03})(v_{21} + v_{03}) \left[3(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2 \right] \\
 h_5 &= (v_{20} - v_{02}) \left[(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2 \right] + 4v_{11}(v_{30} + v_{12})(v_{21} + v_{03}) \\
 h_6 &= (3v_{21} - v_{03})(v_{21} + v_{03}) \left[3(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2 \right] \\
 &\quad - (v_{30} - 3v_{12})(v_{21} + v_{03}) \left[3(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2 \right]
 \end{aligned}$$

本例对 Hu 矩中的第 0 个矩 $h_0 = v_{20} + v_{02}$ 的关系进行验证, 即 Hu 矩中第 0 个矩对应的函数 `cv2.moments()` 的返回值为:

$$h_0 = nu_{20} + nu_{02}$$

根据题目的要求及分析, 编写代码如下:

```
import cv2
o1 = cv2.imread('cs1.bmp')
gray = cv2.cvtColor(o1, cv2.COLOR_BGR2GRAY)
HuM1=cv2.HuMoments(cv2.moments(gray)).flatten()
print("cv2.moments(gray)=\n",cv2.moments(gray))
print("\nHuM1=\n", HuM1)
print("\ncv2.moments(gray)['nu20']+cv2.moments(gray)['nu02']=%f+%f=%f\n"
      %(cv2.moments(gray)['nu20'],cv2.moments(gray)['nu02'],
        cv2.moments(gray)['nu20']+cv2.moments(gray)['nu02']))
print("HuM1[0]=", HuM1[0])
print("\nHu[0]-(nu02+nu20)=",
      HuM1[0]-(cv2.moments(gray)['nu20']+cv2.moments(gray)['nu02']))
```

运行上述程序, 结果如下所示:

```
cv2.moments(gray)=
{'m00': 2729265.0, 'm10': 823361085.0, 'm01': 353802555.0, 'm20':
256058984145.0, 'm11': 104985534390.0, 'm02': 47279854725.0, 'm30':
81917664997185.0, 'm21': 32126275537320.0, 'm12': 13822864338150.0, 'm03':
6484319942535.0, 'mu20': 7668492092.239532, 'mu11': -1749156290.667572, 'mu02':
1415401136.019806, 'mu30': 43285283824.25, 'mu21': -12028503719.705078, 'mu12':
13036213891.871826, 'mu03': -11670178717.88086, 'nu20': 0.0010294815371794503,
'nu11': -0.00023482114674224926, 'nu02': 0.00019001510593064523, 'nu30':
3.5174343862137483e-06, 'nu21': -9.774562821438012e-07, 'nu12':
1.05934444921254788e-06, 'nu03': -9.483381946207041e-07}
```

```
HuM1=
[ 1.21949664e-03  9.25267773e-07  4.05157060e-12  2.46555893e-11
 2.41189094e-22  2.27497012e-14 -5.05282814e-23]
```

```
cv2.moments(gray)['nu20']+cv2.moments(gray)['nu02']=0.001029+0.000190=0.0012
```

19

```
HuM1[0]= 0.0012194966431100956
```

```
Hu[0]-(nu02+nu20)= 0.0
```

程序运行结果显示“`Hu[0]-(nu02+nu20)= 0.0`”。从该结果可知, 关系 $h_0 = nu_{20} + nu_{02}$ 成立。

【例 12.9】 计算三幅不同图像的 Hu 矩, 并进行比较。

根据题目的要求, 编写代码如下:

```
import cv2
#-----计算图像 o1 的 Hu 矩-----
```



```

o1 = cv2.imread('cs1.bmp')
gray1 = cv2.cvtColor(o1,cv2.COLOR_BGR2GRAY)
HuM1=cv2.HuMoments(cv2.moments(gray1)).flatten()
#-----计算图像 o2 的 Hu 矩-----
o2 = cv2.imread('cs3.bmp')
gray2 = cv2.cvtColor(o2,cv2.COLOR_BGR2GRAY)
HuM2=cv2.HuMoments(cv2.moments(gray2)).flatten()
#-----计算图像 o3 的 Hu 矩-----
o3 = cv2.imread('lena.bmp')
gray3 = cv2.cvtColor(o3,cv2.COLOR_BGR2GRAY)
HuM3=cv2.HuMoments(cv2.moments(gray3)).flatten()
#-----打印图像 o1、图像 o2、图像 o3 的特征值-----
print("o1.shape=", o1.shape)
print("o2.shape=", o2.shape)
print("o3.shape=", o3.shape)
print("cv2.moments(gray1)=\n",cv2.moments(gray1))
print("cv2.moments(gray2)=\n",cv2.moments(gray2))
print("cv2.moments(gray3)=\n",cv2.moments(gray3))
print("\nHuM1=\n", HuM1)
print("\nHuM2=\n", HuM2)
print("\nHuM3=\n", HuM3)
#-----计算图像 o1 与图像 o2、图像 o3 的 Hu 矩之差-----
print("\nHuM1-HuM2=", HuM1-HuM2)
print("\nHuM1-HuM3=", HuM1-HuM3)
#-----显示图像-----
cv2.imshow("original1",o1)
cv2.imshow("original2",o2)
cv2.imshow("original3",o3)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示各个图像的 shape 属性、moments 属性、HuMoments 属性，以及不同图像的 Hu 矩之差。

```

o1.shape= (425, 514, 3)
o2.shape= (425, 514, 3)
o3.shape= (512, 512, 3)
cv2.moments(gray1)=
{'m00': 2729265.0, 'm10': 823361085.0, 'm01': 353802555.0, 'm20':
256058984145.0, 'm11': 104985534390.0, 'm02': 47279854725.0, 'm30':
81917664997185.0, 'm21': 32126275537320.0, 'm12': 13822864338150.0, 'm03':
6484319942535.0, 'mu20': 7668492092.239532, 'mu11': -1749156290.667572, 'mu02':
1415401136.019806, 'mu30': 43285283824.25, 'mu21': -12028503719.705078, 'mu12':
13036213891.871826, 'mu03': -11670178717.88086, 'nu20': 0.0010294815371794503,
'nu11': -0.00023482114674224926, 'nu02': 0.00019001510593064523, 'nu30':
3.5174343862137483e-06, 'nu21': -9.774562821438012e-07, 'nu12':
1.0593444921254788e-06, 'nu03': -9.483381946207041e-07}
cv2.moments(gray2)=

```

```
{'m00': 1755675.0, 'm10': 518360685.0, 'm01': 190849140.0, 'm20':
156229722135.0, 'm11': 55624504050.0, 'm02': 21328437150.0, 'm30': 47992502493915.0,
'm21': 16559578863270.0, 'm12': 6135747671370.0, 'm03': 2448843661890.0, 'mu20':
3184426306.518524, 'mu11': -723448129.1111069, 'mu02': 582345624.6666679, 'mu30':
-14508249198.71875, 'mu21': 3955540976.4630127, 'mu12': -4161129804.772583, 'mu03':
3747496072.099121, 'nu20': 0.001033101406743052, 'nu11': -0.00023470327398074646,
'nu02': 0.00018892636416872798, 'nu30': -3.5522595786074026e-06, 'nu21':
9.684909688107435e-07, 'nu12': -1.0188281855633918e-06, 'nu03':
9.175523962659349e-07}
```

```
cv2.moments(gray3)=
```

```
{'m00': 32524520.0, 'm10': 8668693016.0, 'm01': 8048246168.0, 'm20':
3012074835288.0, 'm11': 2188197716912.0, 'm02': 2697437187672.0, 'm30':
1162360702630328.0, 'm21': 771188127583648.0, 'm12': 737629807045152.0, 'm03':
1024874860779368.0, 'mu20': 701626022956.6519, 'mu11': 43115319152.083496, 'mu02':
705885386731.4578, 'mu30': -14447234840442.0, 'mu21': 2862363425762.25, 'mu12':
-2650458863973.0, 'mu03': 8044566997348.25, 'nu20': 0.00066326013744609, 'nu11':
4.075771361264021e-05, 'nu02': 0.0006672865932933317, 'nu30':
-2.3947351703101695e-06, 'nu21': 4.744577382167515e-07, 'nu12':
-4.3933300241295835e-07, 'nu03': 1.3334460006519109e-06}
```

```
HuM1=
```

```
[ 1.21949664e-03 9.25267773e-07 4.05157060e-12 2.46555893e-11
2.41189094e-22 2.27497012e-14 -5.05282814e-23]
```

```
HuM2=
```

```
[ 1.22202777e-03 9.32974010e-07 4.19762083e-12 2.44520029e-11
2.44855011e-22 2.27298009e-14 -3.76120600e-23]
```

```
HuM3=
```

```
[ 1.33054673e-03 6.66097722e-09 1.16744767e-12 1.13004583e-11
-2.02613532e-24 -8.54504575e-16 4.09952009e-23]
```

```
HuM1-HuM2= [-2.53112780e-06 -7.70623675e-09 -1.46050222e-13 2.03586345e-13
-3.66591675e-24 1.99003443e-17 -1.29162214e-23]
```

```
HuM1-HuM3= [-1.11050088e-04 9.18606796e-07 2.88412294e-12 1.33551309e-11
2.43215229e-22 2.36042058e-14 -9.15234823e-23]
```

同时, 还会显示三幅原始图像, 如图 12-14 所示:

- 左图是图像 o1。
- 中间的是图像 o2。
- 右图是图像 o3。



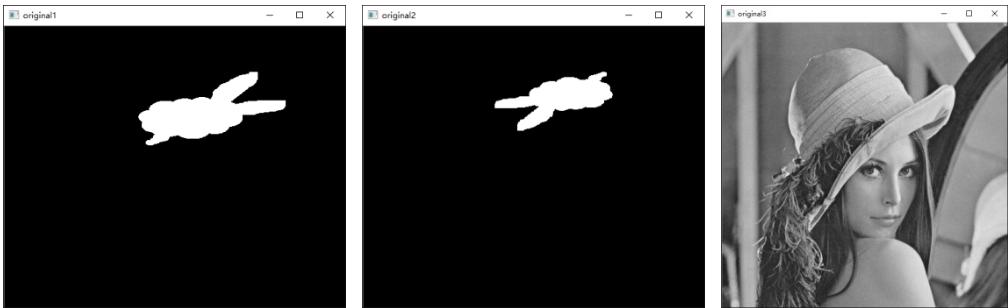


图 12-14 【例 12.9】程序运行的结果

从上述输出结果可以看到，由于 Hu 矩的值本身就非常小，因此在这里并没有发现两个对象的 Hu 矩差值的特殊意义。

12.3.2 形状匹配

我们可以通过 Hu 矩来判断两个对象的一致性。例如，上一节计算了两个对象 Hu 矩的差，但是结果比较抽象。为了更直观方便地比较 Hu 矩值，OpenCV 提供了函数 cv2.matchShapes()，对两个对象的 Hu 矩进行比较。

函数 cv2.matchShapes()允许我们提供两个对象，对二者的 Hu 矩进行比较。这两个对象可以是轮廓，也可以是灰度图。不管是什么，cv2.matchShapes()都会提前计算好对象的 Hu 矩值。

函数 cv2.matchShapes()的语法格式为：

```
retval = cv2.matchShapes( contour1, contour2, method, parameter )
```

式中 retval 是返回值。

该函数有如下 4 个参数：

- contour1：第 1 个轮廓或者灰度图像。
- contour2：第 2 个轮廓或者灰度图像。
- method：比较两个对象的 Hu 矩的方法，具体如表 12-1 所示。

表 12-1 method 的值及其具体含义

方法名称	计算方法
cv2.CONTOURS_MATCH_I1	$\sum_{i=1...7} \left \frac{1}{m_i^A} - \frac{1}{m_i^B} \right $
cv2.CONTOURS_MATCH_I2	$\sum_{i=1...7} m_i^A - m_i^B $
cv2.CONTOURS_MATCH_I3	$\max_{i=1...7} \frac{ m_i^A - m_i^B }{ m_i^A }$

在表 12-1 中，A 表示对象 1，B 表示对象 2：

$$m_i^A = \text{sign}\left(h_i^A\right) \log h_i^A$$
$$m_i^B = \text{sign}\left(h_i^B\right) \log h_i^B$$

式中, h_i^A 和 h_i^B 分别是对象 A 和对象 B 的 Hu 矩。

- parameter: 应用于 method 的特定参数, 该参数为扩展参数, 目前 (截至 OpenCV 4.1.0 版本) 暂不支持该参数, 因此将该值设置为 0。

【例 12.10】 使用函数 cv2.matchShapes() 计算三幅不同图像的匹配度。

根据题目要求, 编写代码如下:

```
import cv2
o1 = cv2.imread('cs1.bmp')
o2 = cv2.imread('cs2.bmp')
o3 = cv2.imread('cc.bmp')
gray1 = cv2.cvtColor(o1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(o2, cv2.COLOR_BGR2GRAY)
gray3 = cv2.cvtColor(o3, cv2.COLOR_BGR2GRAY)
ret, binary1 = cv2.threshold(gray1, 127, 255, cv2.THRESH_BINARY)
ret, binary2 = cv2.threshold(gray2, 127, 255, cv2.THRESH_BINARY)
ret, binary3 = cv2.threshold(gray3, 127, 255, cv2.THRESH_BINARY)
image, contours1, hierarchy = cv2.findContours(binary1,
                                                cv2.RETR_LIST,
                                                cv2.CHAIN_APPROX_SIMPLE)
image, contours2, hierarchy = cv2.findContours(binary2,
                                                cv2.RETR_LIST,
                                                cv2.CHAIN_APPROX_SIMPLE)
image, contours3, hierarchy = cv2.findContours(binary3,
                                                cv2.RETR_LIST,
                                                cv2.CHAIN_APPROX_SIMPLE)

cnt1 = contours1[0]
cnt2 = contours2[0]
cnt3 = contours3[0]
ret0 = cv2.matchShapes(cnt1, cnt1, 1, 0.0)
ret1 = cv2.matchShapes(cnt1, cnt2, 1, 0.0)
ret2 = cv2.matchShapes(cnt1, cnt3, 1, 0.0)
print("相同图像的 matchShape=", ret0)
print("相似图像的 matchShape=", ret1)
print("不相似图像的 matchShape=", ret2)
```

运行上述程序, 会显示三幅原始图像, 如图 12-15 所示。其中:

- 左图是图像 o1。
- 中间的是图像 o2。
- 右图是图像 o3。



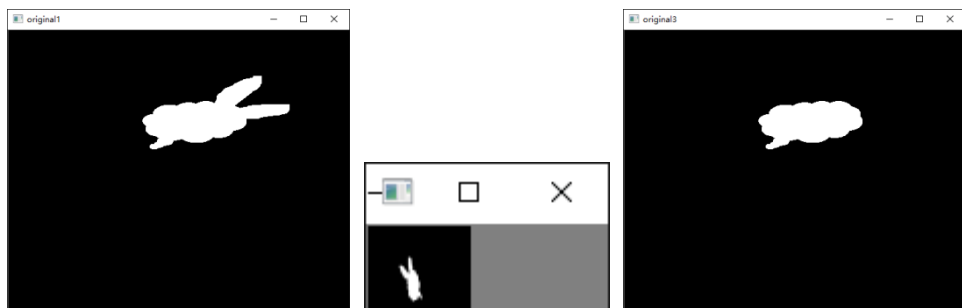


图 12-15 【例 12.10】程序的运行结果

同时，程序还会显示如下运行结果：

```
o1.shape= (425, 514, 3)
o2.shape= (42, 51, 3)
o3.shape= (425, 514, 3)
相同图像的 matchShape= 0.0
相似图像的 matchShape= 0.0001154058519395873
不相似图像的 matchShape= 0.012935752303635306
```

从以上结果可以看出：

- 同一幅图像的 Hu 矩是不变的，二者差值为 0。
- 相似的图像即使发生了平移、旋转和缩放后，函数 `cv2.matchShapes()` 的返回值仍然比较接近。例如，图像 o1 和图像 o2，o2 是对 o1 经过缩放、旋转和平移后得到的，但是对二者应用 `cv2.matchShapes()` 函数后，返回值的差较小。
- 不相似图像 `cv2.matchShapes()` 函数返回值的差较大。例如，图像 o1 和图像 o3 的差别较大，因此对二者应用 `cv2.matchShapes()` 函数后，返回值的差也较大。

12.4 轮廓拟合

在计算轮廓时，可能并不需要实际的轮廓，而仅需要一个接近于轮廓的近似多边形。OpenCV 提供了多种计算轮廓近似多边形的方法。

本节的部分程序使用了绘图函数，这些函数在第 19 章中有详细的介绍，这里不再赘述。

12.4.1 矩形包围框

函数 `cv2.boundingRect()` 能够绘制轮廓的矩形边界。该函数的语法格式为：

```
retval = cv2.boundingRect( array )
```

式中：

- 返回值 `retval` 表示返回的矩形边界的左上角顶点的坐标值及矩形边界的宽度和高度。
- 参数 `array` 是灰度图像或轮廓。

该函数还可以是具有 4 个返回值的形式：

```
x,y,w,h = cv2.boundingRect( array )
```

这里的 4 个返回值分别表示：

- 矩形边界左上角顶点的 x 坐标。
- 矩形边界左上角顶点的 y 坐标。
- 矩形边界的 x 方向的长度。
- 矩形边界的 y 方向的长度。

【例 12.11】设计程序，显示函数 `cv2.boundingRect()` 不同形式的返回值。

根据题目的要求，编写代码如下：

```
import cv2
#-----读取并显示原始图像-----
o = cv2.imread('cc.bmp')
#-----提取图像轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

#-----返回顶点及边长-----
x,y,w,h = cv2.boundingRect(contours[0])
print("顶点及长宽的点形式:")
print("x=",x)
print("y=",y)
print("w=",w)
print("h=",h)
#-----仅有一个返回值的情况-----
rect = cv2.boundingRect(contours[0])
print("\n 顶点及长宽的元组 (tuple) 形式: ")
print("rect=",rect)
```

运行上述程序，显示如下结果：

顶点及长宽的点形式：

```
x= 202
y= 107
w= 157
h= 73
```

顶点及长宽的元组 (tuple) 形式：

```
rect= (202, 107, 157, 73)
```

【例 12.12】使用函数 `cv2.drawContours()` 绘制矩形包围框。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
#-----读取并显示原始图像-----
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
```



```

#-----提取图像轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)

#-----构造矩形边界-----
x,y,w,h = cv2.boundingRect(contours[0])
brcnt = np.array([[x, y]], [[x+w, y]], [[x+w, y+h]], [[x, y+h]]])
cv2.drawContours(o, [brcnt], -1, (255, 255, 255), 2)
#-----显示矩形边界-----
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-16 所示的图像，其中：

- 左图是原始图像 o。
- 右图是在原始图像外绘制矩形包围框后的结果。



图 12-16 【例 12.12】程序的运行结果

【例 12.13】使用函数 `cv2.boundingRect()` 及 `cv2.rectangle()` 绘制矩形包围框。

函数 `cv2.rectangle()` 可以用来绘制矩形，有关该函数的详细介绍可以参考第 19 章。

根据题目的要求，编写代码如下：

```

import cv2
#-----读取并显示原始图像-----
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
#-----提取图像轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)

#-----构造矩形边界-----
x,y,w,h = cv2.boundingRect(contours[0])
cv2.rectangle(o, (x,y), (x+w,y+h), (255, 255, 255), 2)

```

```
#-----显示矩形边界-----
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-17 所示的图像，其中：

- 左图是原始图像 o。
- 右图是在原始图像外绘制矩形包围框后的结果。



图 12-17 【例 12.13】程序的运行结果

12.4.2 最小包围矩形框

函数 `cv2.minAreaRect()` 能够绘制轮廓的最小包围矩形框，其语法格式为：

```
retval = cv2.minAreaRect( points )
```

式中：

- 返回值 `retval` 表示返回的矩形特征信息。
该值的结构是 (最小外接矩形的中心 (x, y) , (宽度, 高度), 旋转角度)。
- 参数 `points` 是轮廓。

需要注意，返回值 `retval` 的结构不符合函数 `cv2.drawContours()` 的参数结构要求。因此，必须将其转换为符合要求的结构，才能使用。函数 `cv2.boxPoints()` 能够将上述返回值 `retval` 转换为符合要求的结构。函数 `cv2.boxPoints()` 的语法格式是：

```
points = cv2.boxPoints( box )
```

式中：

- 返回值 `points`，是能够用于函数 `cv2.drawContours()` 参数的轮廓点。
- 参数 `box` 是函数 `cv2.minAreaRect()` 返回值的类型的值。

【例 12.14】使用函数 `cv2.minAreaRect()` 计算图像的最小包围矩形框。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
```

```

ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

rect = cv2.minAreaRect(contours[0])
print("返回值 rect:\n", rect)
points = cv2.boxPoints(rect)
print("\n 转换后的 points: \n", points)
points = np.int0(points) #取整
image=cv2.drawContours(o, [points], 0, (255, 255, 255), 2)
cv2.imshow("result", o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-18 所示的图像：

- 左图是原始图像 o。
- 右图显示了最小包围矩形框。



图 12-18 【例 12.14】程序的运行结果

同时，程序还会输出如下结果：

返回值 rect：

```
((280.3699951171875, 138.58999633789062), (154.99778747558594,
63.78103256225586), -8.130102157592773)
```

转换后的 points：

```
[[208.16002  181.12    ]
 [199.14    117.979996]
 [352.57996  96.06    ]
 [361.59998  159.2    ]]
```

从输出结果可以看出：

- 返回值 rect 表示返回的矩形特征信息。
该值的结构是(最小外接矩形的中心(x,y),(宽度,高度),旋转角度)
- 转换后的 points 是一些点，是能够用作函数 cv2.drawContours() 参数的轮廓点。

12.4.3 最小包围圆形

函数 `cv2.minEnclosingCircle()` 通过迭代算法构造一个对象的面积最小包围圆形。该函数的语法格式为：

```
center, radius = cv2.minEnclosingCircle( points )
```

式中：

- 返回值 `center` 是最小包围圆形的中心。
- 返回值 `radius` 是最小包围圆形的半径。
- 参数 `points` 是轮廓。

【例 12.15】 使用函数 `cv2.minEnclosingCircle()` 构造图像的最小包围圆形。

根据题目的要求，编写代码如下：

```
import cv2
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)
(x,y),radius = cv2.minEnclosingCircle(contours[0])
center = (int(x),int(y))
radius = int(radius)
cv2.circle(o,center,radius,(255,255,255),2)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例中调用了函数 `cv2.circle()`，有关该函数的详细说明请参考第 19 章。

运行上述程序，会显示如图 12-19 所示的图像。其中：

- 左图是图像 `o`。
- 右图是含有最小包围圆形的图像。



图 12-19 【例 12.15】程序的运行结果

12.4.4 最优拟合椭圆

在 OpenCV 中，函数 `cv2.fitEllipse()` 可以用来构造最优拟合椭圆。该函数的语法格式是：

```
retval = cv2.fitEllipse( points )
```

式中：

- 返回值 `retval` 是 `RotatedRect` 类型的值。这是因为该函数返回的是拟合椭圆的外接矩形，`retval` 包含外接矩形的质心、宽、高、旋转角度等参数信息，这些信息正好与椭圆的中心点、轴长度、旋转角度等信息吻合。
- 参数 `points` 是轮廓。

【例 12.16】 使用函数 `cv2.fitEllipse()` 构造最优拟合椭圆。

本例需要使用函数 `cv2.ellipse()` 根据函数 `cv2.fitEllipse()` 的返回值绘制最优拟合椭圆。有关函数 `cv2.ellipse()` 的用法，请参考第 19 章。

根据题目的要求，编写代码如下：

```
import cv2
o = cv2.imread('cc.bmp')
gray = cv2.cvtColor(o, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cv2.imshow("original", o)
ellipse = cv2.fitEllipse(contours[0])
print("ellipse=", ellipse)
cv2.ellipse(o, ellipse, (0, 255, 0), 3)
cv2.imshow("result", o)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-20 所示的图像。其中：

- 左图是图像 `o`。
- 右图是包含了最优拟合椭圆的图像。



图 12-20 【例 12.16】程序的运行结果

同时，程序还会显示如下的运行结果：

```
ellipse= ((276.2112731933594, 139.6067352294922), (63.01350021362305,
166.72308349609375), 82.60102844238281)
```

上述运行结果中：

- (276.2112731933594, 139.6067352294922)是椭圆的中心点。
- (63.01350021362305, 166.72308349609375)是椭圆的轴长度。
- 82.60102844238281 是椭圆的旋转角度。

12.4.5 最优拟合直线

在 OpenCV 中，函数 cv2.fitLine()用来构造最优拟合直线，该函数的语法格式为：

```
line = cv2.fitLine( points, distType, param, reps, aeps )
```

式中 line 为返回值，是返回的最优拟合直线参数。

式中的参数如下：

- points: 轮廓。
- distType: 距离类型。拟合直线时，要使输入点到拟合直线的距离之和最小，其类型如表 12- 2 所示。
- param: 距离参数，与所选的距离类型有关。当此参数被设置为 0 时，该函数会自动选择最优值。
- reps: 用于表示拟合直线所需要的径向精度，通常该值被设定为 0.01。
- aeps: 用于表示拟合直线所需要的角度精度，通常该值被设定为 0.01。

表 12-2 距离参数的值以及含义

值	含义（以点(x1,y1)和点(x2,y2)为例）
cv2.DIST_USER	用户定义距离
cv2.DIST_L1	$distance = x1-x2 + y1-y2 $
cv2.DIST_L2	欧氏距离
cv2.DIST_C	$distance = \max(x1-x2 , y1-y2)$
cv2.DIST_L12	$distance = 2(\sqrt{1+x*x/2} - 1)$
cv2.DIST_FAIR	$distance = c2(x /c - \log(1+ x /c))$, $c = 1.3998$
cv2.DIST_WELSCH	$distance = c^2/2(1 - \exp(-(x/c)^2))$, $c = 2.9846$
cv2.DIST_HUBER	$distance = x < c ? x^2/2 : c(x - c/2)$, $c = 1.345$

【例 12.17】使用函数 cv2.fitLine()构造最优拟合直线。

根据题目的要求，编写代码如下：

```
import cv2
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
```

```

cv2.RETR_LIST,
cv2.CHAIN_APPROX_SIMPLE)

rows,cols = image.shape[:2]
[vx,vy,x,y] = cv2.fitLine(contours[0], cv2.DIST_L2,0,0.01,0.01)
lefty = int((-x*vy/vx) + y)
righty = int(((cols-x)*vy/vx)+y)
cv2.line(o,(cols-1,righty),(0,lefty),(0,255,0),2)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-21 所示的图像。其中：

- 左图是图像 o。
- 右图是包含了最优拟合直线的图像。



图 12-21 【例 12.17】程序的运行结果

12.4.6 最小外包三角形

在 OpenCV 中，函数 `cv2.minEnclosingTriangle()` 用来构造最小外包三角形。该函数的语法格式为：

```
retval, triangle = cv2.minEnclosingTriangle( points )
```

式中有两个返回值：

- **retval**：最小外包三角形的面积。
- **triangle**：最小外包三角形的三个顶点集。

式中的参数 `points` 是轮廓。

【例 12.18】使用函数 `cv2.minEnclosingTriangle()` 构造最小外包三角形。

根据题目的要求，编写代码如下：

```

import cv2
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,

```

```

cv2.RETR_LIST,
cv2.CHAIN_APPROX_SIMPLE)
area, trgl = cv2.minEnclosingTriangle(contours[0])
print("area=", area)
print("trgl:", trgl)
for i in range(0, 3):
    cv2.line(o, tuple(trgl[i][0]),
             tuple(trgl[(i + 1) % 3][0]), (255, 255, 255), 2)
cv2.imshow("result", o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-22 所示的图像。其中：

- 左图是图像 o。
- 右图是包含最小外包三角形的图像。



图 12-22 【例 12.18】程序的运行结果

同时，程序还会显示如下的运行结果：

```

area= 12904.00390625
trgl: [[193.25641 107.    ]
       [[222.58975 211.    ]
       [[441.41025 107.    ]]

```

从上述结果可以看出：

- 返回值 area 是最小外包三角形的面积。
- 返回值 trgl 是最小外包三角形的三个顶点。

12.4.7 逼近多边形

函数 `cv2.approxPolyDP()` 用来构造指定精度的逼近多边形曲线。该函数的语法格式为：

```
approxCurve = cv2.approxPolyDP( curve, epsilon, closed )
```

式中，返回值 `approxCurve` 为逼近多边形的点集。

式中的参数如下：

- `curve` 是轮廓。
- `epsilon` 为精度，原始轮廓的边界点与逼近多边形边界之间的最大距离。
- `closed` 是布尔型值。该值为 `True` 时，逼近多边形是封闭的；否则，逼近多边形是不封闭的。

函数 `cv2.approxPolyDP()` 采用的是 Douglas-Peucker 算法（DP 算法）。以图 12-23 为例，该算法首先从轮廓中找到距离最远的两个点，并将两点相连（见图 12-23 的(b)图）。接下来，在轮廓上找到一个离当前直线最远的点，并将该点与原有直线连成一个封闭的多边形，此时得到一个三角形，如图 12-23 中的(c)图所示。

将上述过程不断地迭代，将新找到的距离当前多边形最远的点加入到结果中。当轮廓上所有的点到当前多边形的距离都小于函数 `cv2.approxPolyDP()` 的参数 `epsilon` 的值时，就停止迭代。最终可以得到如图 12-23 的(f)图所示的处理结果。

通过上述过程可知，`epsilon` 是逼近多边形的精度信息。通常情况下，将该精度设置为多边形总长度的百分比形式。

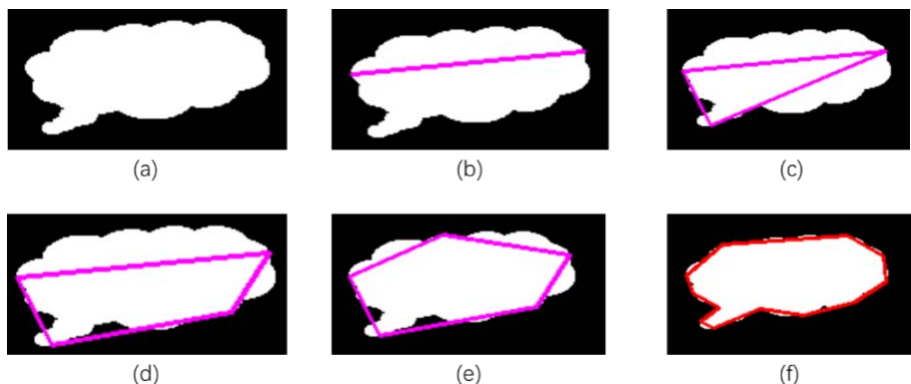


图 12-23 DP 算法

【例 12.19】 使用函数 `cv2.approxPolyDP()` 构造不同精度的逼近多边形。

根据题目的要求，编写代码如下：

```
import cv2
#-----读取并显示原始图像-----
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
#-----获取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                             cv2.RETR_LIST,
                                             cv2.CHAIN_APPROX_SIMPLE)
#-----epsilon=0.1*周长-----
adp = o.copy()
epsilon = 0.1*cv2.arcLength(contours[0],True)
```

```

approx = cv2.approxPolyDP(contours[0],epsilon,True)
adp=cv2.drawContours(adp, [approx],0,(0,0,255),2)
cv2.imshow("result0.1",adp)
#-----epsilon=0.09*周长-----
adp = o.copy()
epsilon = 0.09*cv2.arcLength(contours[0],True)
approx = cv2.approxPolyDP(contours[0],epsilon,True)
adp=cv2.drawContours(adp, [approx],0,(0,0,255),2)
cv2.imshow("result0.09",adp)
#-----epsilon=0.055*周长-----
adp = o.copy()
epsilon = 0.055*cv2.arcLength(contours[0],True)
approx = cv2.approxPolyDP(contours[0],epsilon,True)
adp=cv2.drawContours(adp, [approx],0,(0,0,255),2)
cv2.imshow("result0.055",adp)
#-----epsilon=0.05*周长-----
adp = o.copy()
epsilon = 0.05*cv2.arcLength(contours[0],True)
approx = cv2.approxPolyDP(contours[0],epsilon,True)
adp=cv2.drawContours(adp, [approx],0,(0,0,255),2)
cv2.imshow("result0.05",adp)
#-----epsilon=0.02*周长-----
adp = o.copy()
epsilon = 0.02*cv2.arcLength(contours[0],True)
approx = cv2.approxPolyDP(contours[0],epsilon,True)
adp=cv2.drawContours(adp, [approx],0,(0,0,255),2)
cv2.imshow("result0.02",adp)
#-----等待释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-24 所示的图像。其中：

- (a)图是图像 o。
- (b)图显示了“ $\epsilon=0.1 \times \text{周长}$ ”的逼近多边形（实际上，在第 1 次的查找过程中仅仅找到了一条直线）。
- (c)图显示了“ $\epsilon=0.09 \times \text{周长}$ ”的逼近多边形。
- (d)图显示了“ $\epsilon=0.055 \times \text{周长}$ ”的逼近多边形。
- (e)图显示了“ $\epsilon=0.05 \times \text{周长}$ ”的逼近多边形。
- (f)图显示了“ $\epsilon=0.02 \times \text{周长}$ ”的逼近多边形。



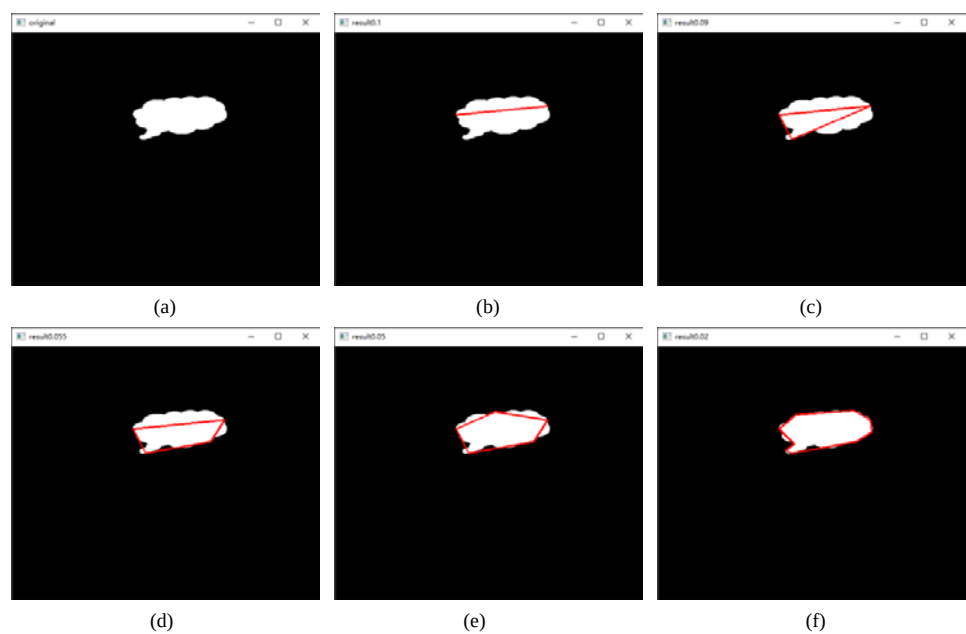


图 12-24 【例 12.19】程序的运行结果

从程序运行结果可以看出，在函数 `cv2.approxPolyDP()` 中通过参数 `epsilon` 可以控制逼近多边形的精度。

需要注意：本节中使用的图像都是仅有一个轮廓的图像，处理的轮廓都是 `contours[0]`。如果处理的原图像中有多个轮廓，则需要注意控制轮廓的索引，即 `contours[i]` 中的 `i` 值，使其指向特定的轮廓。

12.5 凸包

逼近多边形是轮廓的高度近似，但是有时候，我们希望使用一个多边形的凸包来简化它。凸包跟逼近多边形很像，只不过它是物体最外层的“凸”多边形。凸包指的是完全包含原有轮廓，并且仅由轮廓上的点所构成的多边形。凸包的每一处都是凸的，即在凸包内连接任意两点的直线都在凸包的内部。在凸包内，任意连续三个点的内角小于 180° 。

例如，在图 12-25 中，最外层的多边形为机械手的凸包，在机械手边缘与凸包之间的部分被称为凸缺陷（Convexity Defect），凸缺陷能够用来处理手势识别等问题。



图 12-25 凸包示意图

12.5.1 获取凸包

OpenCV 提供函数 `cv2.convexHull()` 用于获取轮廓的凸包。该函数的语法格式为:

```
hull = cv2.convexHull( points[, clockwise[, returnPoints]] )
```

式中的返回值 `hull` 为凸包角点。

式中的参数如下:

- `points`: 轮廓。
- `clockwise`: 布尔型值。该值为 `True` 时, 凸包角点将按顺时针方向排列; 该值为 `False` 时, 则以逆时针方向排列凸包角点。
- `returnPoints`: 布尔型值。默认值是 `True`, 函数返回凸包角点的 x/y 轴坐标; 当为 `False` 时, 函数返回轮廓中凸包角点的索引。

【例 12.20】 设计程序, 观察函数 `cv2.convexHull()` 内参数 `returnPoints` 的使用情况。

根据题目的要求, 编写代码如下:

```
import cv2
o = cv2.imread('contours.bmp')
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_TREE,
                                              cv2.CHAIN_APPROX_SIMPLE)

hull = cv2.convexHull(contours[0]) # 返回坐标值
print("returnPoints 为默认值 True 时返回值 hull 的值: \n",hull)
hull2 = cv2.convexHull(contours[0], returnPoints=False) # 返回索引值
print("returnPoints 为 False 时返回值 hull 的值: \n",hull2)
```

运行上述程序, 显示如下的结果:

`returnPoints` 为默认值 `True` 时返回值 `hull` 的值:

```
[[[195 383]]
```

```
[[ 79 383]]
```

```
[[ 79 270]]
```

```
[[195 270]]]
```

`returnPoints` 为 `False` 时返回值 `hull` 的值:

```
[[2]
```

```
[1]
```

```
[0]
```

```
[3]]
```

从程序运行结果可以看出, 函数 `cv2.convexHull()` 的可选参数 `returnPoints`:

- 为默认值 `True` 时, 函数返回凸包角点的 x/y 轴坐标, 本例中返回了 4 个轮廓的坐标值。
- 为 `False` 时, 函数返回轮廓中凸包角点的索引, 本例中返回了 4 个轮廓的索引值。



【例 12.21】使用函数 `cv2.convexHull()` 获取轮廓的凸包。

根据题目的要求，编写代码如下：

```
import cv2
# -----读取并绘制原始图像-----
o = cv2.imread('hand.bmp')
cv2.imshow("original",o)
# -----提取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

# -----寻找凸包，得到凸包的角点-----
hull = cv2.convexHull(contours[0])
# -----绘制凸包-----
cv2.polylines(o, [hull], True, (0, 255, 0), 2)
# -----显示凸包-----
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-26 所示的图像。其中：

- 左图是图像 `o`。
- 右图是包含获取的凸包的图像。

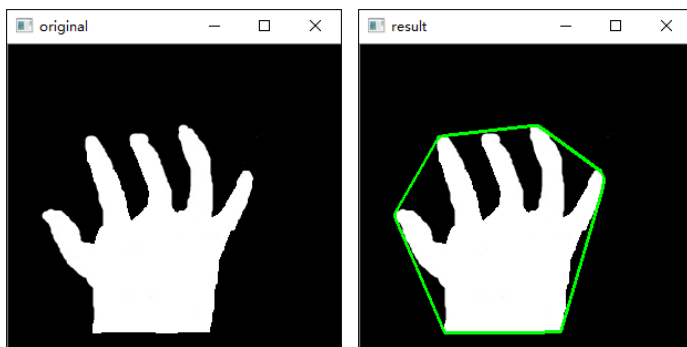


图 12-26 【例 12.21】程序的运行结果

12.5.2 凸缺陷

凸包与轮廓之间的部分，称为凸缺陷。在 OpenCV 中使用函数 `cv2.convexityDefects()` 获取凸缺陷。其语法格式如下：

```
convexityDefects = cv2.convexityDefects( contour, convexhull )
```

式中的返回值 `convexityDefects` 为凸缺陷点集。它是一个数组，每一行包含的值是[起点，终点，轮廓上距离凸包最远的点，最远点到凸包的近似距离]。

需要注意的是，返回结果中[起点，终点，轮廓上距离凸包最远的点，最远点到凸包的近似距离]的前三个值是轮廓点的索引，所以需要到轮廓点中去找它们。

式中的参数如下：

- `contour` 是轮廓。
- `convexhull` 是凸包。

需要注意的是，用 `cv2.convexityDefects()` 计算凸缺陷时，要使用凸包作为参数。在查找该凸包时，所使用函数 `cv2.convexHull()` 的参数 `returnPoints` 的值必须是 `False`。

为了让大家更直观地观察凸缺陷点集，我们尝试将凸缺陷点集在一幅图内显示出来。实现方式为，将起点和终点用一条线连接，在最远点画一个圆圈。下面我们通过一个例子来展示上述操作。

【例 12.22】使用函数 `cv2.convexityDefects()` 计算凸缺陷。

根据题目的要求，编写代码如下：

```
import cv2
#-----原图-----
img = cv2.imread('hand.bmp')
cv2.imshow('original',img)
#-----构造轮廓-----
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 127, 255,0)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_TREE,
                                              cv2.CHAIN_APPROX_SIMPLE)
#-----凸包-----
cnt = contours[0]
hull = cv2.convexHull(cnt,returnPoints = False)
defects = cv2.convexityDefects(cnt,hull)
print("defects=\n",defects)
#-----构造凸缺陷-----
for i in range(defects.shape[0]):
    s,e,f,d = defects[i,0]
    start = tuple(cnt[s][0])
    end = tuple(cnt[e][0])
    far = tuple(cnt[f][0])
    cv2.line(img,start,end,[0,0,255],2)
    cv2.circle(img,far,5,[255,0,0],-1)
#-----显示结果，释放图像-----
cv2.imshow('result',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行上述程序，会显示如下结果：

defects=



```

[[[ 305  311  306  114]]

[[ 311  385  342 13666]]

[[ 385  389  386  395]]

[[ 389  489  435 20327]]

[[   0  102   51 21878]]

[[ 103  184  150 13876]]

[[ 185  233  220  4168]]

[[ 233  238  235   256]]

[[ 238  240  239   247]]

[[ 240  294  255 2715]]

[[ 294  302  295   281]]

[[ 302  304  303   217]]]

```

同时，程序还会显示如图 12-27 所示的图像。其中：

- 左图为图像 o。
- 右图中用点标出了凸缺陷。可以看出，除了在机械手各个手指的指缝间有凸缺陷外，在无名指、小拇指及手的最下端也都有凸缺陷。

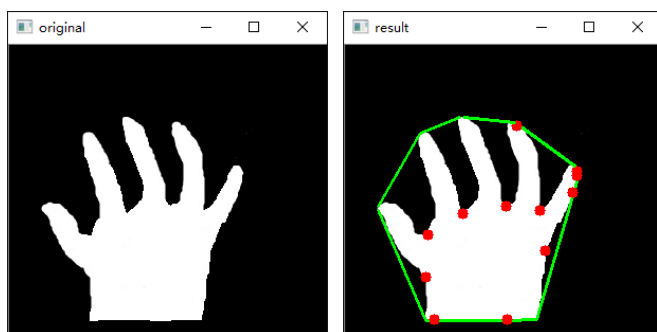


图 12-27 【例 12.22】程序的运行结果

12.5.3 几何学测试

本节介绍几种与凸包有关的几何学测试。

1. 测试轮廓是否是凸形的

在 OpenCV 中，可以用函数 `cv2.isContourConvex()` 来判断轮廓是否是凸形的，其语法格式为：

```
retval = cv2.isContourConvex( contour )
```

式中:

- 返回值 `retval` 是布尔型值。该值为 `True` 时, 表示轮廓为凸形的; 否则, 不是凸形的。
- 参数 `contour` 为要判断的轮廓。

【例 12.23】 使用函数 `cv2.isContourConvex()` 来判断轮廓是否是凸形的。

根据题目的要求, 编写代码如下:

```
import cv2
o = cv2.imread('hand.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

#-----凸包-----
image1=o.copy()
hull = cv2.convexHull(contours[0])
cv2.polylines(image1, [hull], True, (0, 255, 0), 2)
print("使用函数 cv2.convexHull()构造的多边形是否是凸形的: ",
      cv2.isContourConvex(hull))
cv2.imshow("result1",image1)
#-----逼近多边形-----
image2=o.copy()
epsilon = 0.01*cv2.arcLength(contours[0],True)
approx = cv2.approxPolyDP(contours[0],epsilon,True)
image2=cv2.drawContours(image2,[approx],0,(0,0,255),2)
print("使用函数 cv2.approxPolyDP()构造的多边形是否是凸形的: ",
      cv2.isContourConvex(approx))
cv2.imshow("result2",image2)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序, 会显示如图 12-28 所示的图像。其中:

- 左图是图像 `o`。
- 中间的图显示了在图像 `o` 上使用函数 `cv2.convexHull()` 构造的凸包。
- 右图显示了在图像 `o` 上使用函数 `cv2.approxPolyDP()` 构造的逼近多边形。



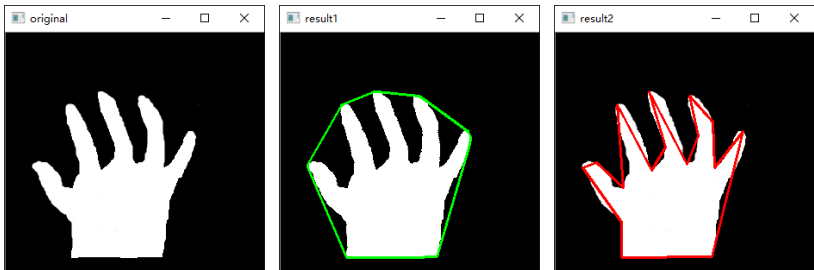


图 12-28 【例 12.23】程序的运行结果

同时，程序还会显示如下的结果：

使用函数 `cv2.convexHull()` 构造的多边形是否是凸形的：True
使用函数 `cv2.approxPolyDP()` 构造的多边形是否是凸形的：False

从以上运行结果可以看出：

- 使用函数 `cv2.convexHull()` 构造凸包后，对绘制的凸包使用函数 `cv2.isContourConvex()` 判断，返回值为 True，说明该轮廓是凸形的。
- 使用函数 `cv2.approxPolyDP()` 构造逼近多边形后，对绘制的逼近多边形使用函数 `cv2.isContourConvex()` 判断，返回值为 False，说明该轮廓（多边形）不是凸形的。

2. 点到轮廓的距离

在 OpenCV 中，函数 `cv2.pointPolygonTest()` 被用来计算点到多边形（轮廓）的最短距离（也就是垂线距离），这个计算过程又称点和多边形的关系测试。该函数的语法格式为：

`retval = cv2.pointPolygonTest( contour, pt, measureDist )`

式中的返回值为 `retval`，与参数 `measureDist` 的值有关。

式中的参数如下：

- `contour` 为轮廓。
- `pt` 为待判定的点。
- `measureDist` 为布尔型值，表示距离的判定方式。
 - 当值为 True 时，表示计算点到轮廓的距离。如果点在轮廓的外部，返回值为负数；如果点在轮廓上，返回值为 0；如果点在轮廓内部，返回值为正数。
 - 当值为 False 时，不计算距离，只返回“-1”、“0”和“1”中的一个值，表示点相对于轮廓的位置关系。如果点在轮廓的外部，返回值为“-1”；如果点在轮廓上，返回值为“0”；如果点在轮廓内部，返回值为“1”。

【例 12.24】使用函数 `cv2.pointPolygonTest()` 计算点到轮廓的最短距离。

使用函数 `cv2.pointPolygonTest()` 计算点到轮廓的最短距离，需要将参数 `measureDist` 的值设置为 True。

根据题目的要求及分析，编写代码如下：

```
import cv2
#-----原始图像-----
o = cv2.imread('cs.bmp')
```

```

cv2.imshow("original",o)
#-----获取凸包-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)

hull = cv2.convexHull(contours[0])
image = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
cv2.polylines(image, [hull], True, (0, 255, 0), 2)
#-----内部点 A 到轮廓的距离-----
distA = cv2.pointPolygonTest(hull, (300, 150), True)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'A', (300,150), font, 1, (0,255,0), 3)
print("distA=",distA)
#-----外部点 B 到轮廓的距离-----
distB = cv2.pointPolygonTest(hull, (300, 250), True)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'B', (300,250), font, 1, (0,255,0), 3)
print("distB=",distB)
#-----正好处于轮廓上的点 C 到轮廓的距离-----
distC = cv2.pointPolygonTest(hull, (423, 112), True)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'C', (423,112), font, 1, (0,255,0), 3)
print("distC=",distC)
#print(hull)  #测试边缘到底在哪里, 然后再使用确定位置的点绘制文字
#-----显示-----
cv2.imshow("result1",image)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序, 会显示如图 12-29 所示的图像。其中:

- 左图是图像 o。
- 右图是标注了点 A、B、C 位置的图像。



图 12-29 【例 12.24】程序的运行结果

同时，程序还会显示如下的结果：

```
distA= 16.891650862259112
distB= -81.17585848021565
distC= -0.0
```

从以上结果可以看出，

- A 点算出来的距离为“16.891650862259112”，是一个正数，说明 A 点在轮廓内部。
- B 点算出来的距离为“-81.17585848021565”，是一个负数，说明 B 点在轮廓外部。
- C 点算出来的距离为“-0.0”，说明 C 点在轮廓上。

在实际使用中，如果想获取位于轮廓上的点，可以通过打印轮廓点集的方式获取。例如，本例中可以通过语句“`print(hull)`”获取轮廓上的点。在获取轮廓上的点以后，可以将其用作函数 `cv2.pointPolygonTest()` 的参数，以测试函数返回值是否为零。

【例 12.25】 使用函数 `cv2.pointPolygonTest()` 判断点与轮廓的关系。

使用函数 `cv2.pointPolygonTest()` 判断点与轮廓的关系时，需要将参数 `measureDist` 的值设置为 `False`。

根据题目的要求及分析，编写代码如下：

```
import cv2
#-----原始图像-----
o = cv2.imread('cs.bmp')
cv2.imshow("original",o)
#-----获取凸包-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)

hull = cv2.convexHull(contours[0])
image = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
cv2.polylines(image, [hull], True, (0, 255, 0), 2)
#-----内部点 A 与轮廓的关系-----
distA = cv2.pointPolygonTest(hull, (300, 150), False)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'A', (300,150), font, 1, (0,255,0), 3)
print("distA=",distA)
#-----外部点 B 与轮廓的关系-----
distB = cv2.pointPolygonTest(hull, (300, 250), False)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'B', (300,250), font, 1, (0,255,0), 3)
print("distB=",distB)
#-----边缘线上的点 C 与轮廓的关系-----
distC = cv2.pointPolygonTest(hull, (423, 112), False)
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(image, 'C', (423,112), font, 1, (0,255,0), 3)
```

```

print("distC=",distC)
#print(hull)  #测试边缘到底在哪里，然后再使用确定位置的点绘制文字
#-----显示-----
cv2.imshow("result",image)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-30 所示的图像。其中：

- 左图是图像 o。
- 右图是标注了点 A、B、C 位置的图像。



图 12-30 【例 12.25】程序的运行结果

同时，程序还会显示如下的运行结果：

```

distA= 1.0
distB= -1.0
distC= 0.0

```

从以上结果可以看出，

- A 点算出来的关系值为“1”，说明该点在轮廓的内部。
- B 点算出来的关系值为“-1”，说明该点在轮廓的外部。
- C 点算出来的关系值为零值，说明该点在轮廓上。

12.6 利用形状场景算法比较轮廓

用矩比较形状是一种非常有效的方法，不过现在有了更有效的方法。从 OpenCV 3 开始，有了专有模块 `shape`，该模块中的形状场景算法能够更高效地比较形状。

12.6.1 计算形状场景距离

OpenCV 提供了使用“距离”作为形状比较的度量标准。这是因为形状之间的差异值和距离有相似之处，比如二者都只能是零或者正数，又比如当两个形状一模一样时距离值和差值都等于零。

OpenCV 提供了函数 `cv2.createShapeContextDistanceExtractor()`，用于计算形状场景距离。



其使用的“形状上下文算法”在计算距离时，在每个点上附加一个“形状上下文”描述符，让每个点都能够捕获剩余点相对于它的分布特征，从而提供全局鉴别特征。

有关该函数的更多理论知识，可以参考学者 Belongie 等人 2002 年在 *IEEE Transactions on Pattern Analysis & Machine Intelligence* 上发表的论文 *Shape Matching and Object Recognition Using Shape Contexts*。

函数 `cv2.createShapeContextDistanceExtractor()` 的语法格式为：

```
retval = cv2.createShapeContextDistanceExtractor(
    [, nAngularBins[,
    nRadialBins[,
    innerRadius[,
    outerRadius[,
    iterations[,
    comparer[,
    transformer]]]]]] )
```

式中的返回值为 `retval`，返回结果。

该结果可以通过函数 `cv2.ShapeDistanceExtractor.computeDistance()` 计算两个不同形状之间的距离。此函数的语法格式为：

```
retval=cv2.ShapeDistanceExtractor.computeDistance(contour1, contour2)
```

式中，`coutour1` 和 `coutour2` 是不同的轮廓。

函数 `cv2.createShapeContextDistanceExtractor()` 的参数都是可选参数：

- `nAngularBins`：为形状匹配中使用的形状上下文描述符建立的角容器的数量。
- `nRadialBins`：为形状匹配中使用的形状上下文描述符建立的径向容器的数量。
- `innerRadius`：形状上下文描述符的内半径。
- `outerRadius`：形状上下文描述符的外半径。
- `iterations`：迭代次数。
- `comparer`：直方图代价提取算子。该函数使用了直方图代价提取仿函数，可以直接采用直方图代价提取仿函数的算子作为参数。
- `transformer`：形状变换参数。

【例 12.26】 使用函数 `cv2.createShapeContextDistanceExtractor()` 计算形状场景距离。

根据题目要求，编写代码如下：

```
import cv2
#-----原始图像 o1 的边缘-----
o1 = cv2.imread('cs.bmp')
cv2.imshow("original1",o1)
gray1 = cv2.cvtColor(o1,cv2.COLOR_BGR2GRAY)
ret, binary1 = cv2.threshold(gray1,127,255,cv2.THRESH_BINARY)
image,contours1, hierarchy = cv2.findContours(binary1,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)
```



```

cnt1 = contours1[0]
#-----原始图像 o2 的边缘-----
o2 = cv2.imread('cs3.bmp')
cv2.imshow("original2",o2)
gray2 = cv2.cvtColor(o2,cv2.COLOR_BGR2GRAY)
ret, binary2 = cv2.threshold(gray2,127,255,cv2.THRESH_BINARY)
image,contours2, hierarchy = cv2.findContours(binary2,
                                          cv2.RETR_LIST,
                                          cv2.CHAIN_APPROX_SIMPLE)

cnt2 = contours2[0]
#-----原始图像 o3 的边缘-----
o3 = cv2.imread('hand.bmp')
cv2.imshow("original3",o3)
gray3 = cv2.cvtColor(o3,cv2.COLOR_BGR2GRAY)
ret, binary3 = cv2.threshold(gray3,127,255,cv2.THRESH_BINARY)
image,contours3, hierarchy = cv2.findContours(binary3,
                                          cv2.RETR_LIST,
                                          cv2.CHAIN_APPROX_SIMPLE)

cnt3 = contours3[0]
#-----构造距离提取算子-----
sd = cv2.createShapeContextDistanceExtractor()
#-----计算距离-----
d1 = sd.computeDistance(cnt1,cnt1)
print("与自身的距离 d1=", d1)
d2 = sd.computeDistance(cnt1,cnt2)
print("与旋转缩放后的自身图像的距离 d2=", d2)
d3 = sd.computeDistance(cnt1,cnt3)
print("与不相似对象的距离 d3=", d3)
#-----显示距离-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-31 所示的图像。其中：

- 左图是图像 o1。
- 中间的是图像 o2。
- 右图是图像 o3。

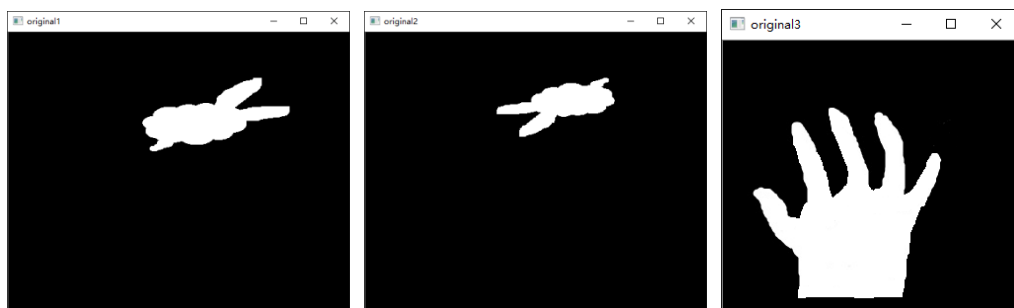


图 12-31 【例 12.26】程序的运行结果



同时，程序还会显示如下的运行结果：

与自身的距离 $d1= 0.0$

与旋转缩放后的自身图像的距离 $d2= 0.7913379669189453$

与不相似对象的距离 $d3= 2.75199031829834$

从上述运行结果可以看出：

- 相同图像之间的形状场景距离为零。
- 相似图像之间的形状场景距离较小。
- 不同图像之间的形状场景距离较大。

12.6.2 计算Hausdorff距离

Hausdorff 距离的计算方法是：

(1) 针对图像 A 内的每一个点，寻找其距离图像 B 的最短距离，将这个最短距离作为 Hausdorff 直接距离 D1。

(2) 针对图像 B 内的每一个点，寻找其距离图像 A 的最短距离，将这个最短距离作为 Hausdorff 直接距离 D2。

(3) 将上述 D1、D2 中的较大者作为 Hausdorff 距离。

通常情况下，Hausdorff 距离 $H(\cdot)$ 是根据对象 A 和对象 B 之间的 Hausdorff 直接距离 $h(\cdot)$ 来定义的，用数学公式表示如下：

$$H(A, B) = \max(h(A, B), h(B, A))$$

其中：

$$h(A, B) = \max_{a \in A} \min_{b \in B} \|a - b\|$$

式中， $\|\cdot\|$ 表示点 a 和点 b 的某种范数，通常是欧氏距离。

麦吉尔大学的学者 Normand 和 Mikael 对 Hausdorff 距离给出了详细的说明，网址为：<http://cgm.cs.mcgill.ca/~godfried/teaching/cg-projects/98/normand/main.html>。如果读者有兴趣的话，可以进一步参考。

OpenCV 提供了函数 `cv2.createHausdorffDistanceExtractor()` 来计算 Hausdorff 距离。其语法格式为：

```
retval = cv2.createHausdorffDistanceExtractor( [, distanceFlag [, rankProp]])
```

式中的返回值 `retval` 为函数返回的结果。

式中的参数如下：

- `distanceFlag` 为距离标记，是可选参数。
- `rankProp` 为一个比例值，范围在 0 到 1 之间，也是可选参数。

【例 12.27】使用函数 `cv2.createHausdorffDistanceExtractor()` 计算不同图像的 Hausdorff 距离。

根据题目的要求，编写代码如下：

```

import cv2
#-----读取原始图像-----
o1 = cv2.imread('cs.bmp')
o2 = cv2.imread('cs3.bmp')
o3 = cv2.imread('hand.bmp')
cv2.imshow("original1",o1)
cv2.imshow("original2",o2)
cv2.imshow("original3",o3)
#-----色彩转换-----
gray1 = cv2.cvtColor(o1,cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(o2,cv2.COLOR_BGR2GRAY)
gray3 = cv2.cvtColor(o3,cv2.COLOR_BGR2GRAY)
#-----阈值处理-----
ret, binary1 = cv2.threshold(gray1,127,255,cv2.THRESH_BINARY)
ret, binary2 = cv2.threshold(gray2,127,255,cv2.THRESH_BINARY)
ret, binary3 = cv2.threshold(gray3,127,255,cv2.THRESH_BINARY)
#-----提取轮廓-----
image,contours1, hierarchy = cv2.findContours(binary1,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)
image,contours2, hierarchy = cv2.findContours(binary2,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)
image,contours3, hierarchy = cv2.findContours(binary3,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cnt1 = contours1[0]
cnt2 = contours2[0]
cnt3 = contours3[0]
#-----构造距离提取算子-----
hd = cv2.createHausdorffDistanceExtractor()
#-----计算距离-----
d1 = hd.computeDistance(cnt1,cnt1)
print("与自身图像的 Hausdorff 距离 d1=", d1)
d2 = hd.computeDistance(cnt1,cnt2)
print("与旋转缩放后的自身图像的 Hausdorff 距离 d2=", d2)
d3 = hd.computeDistance(cnt1,cnt3)
print("与不相似对象的 Hausdorff 距离 d3=", d3)
#-----显示距离-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-32 所示的图像。其中：

- 左图是图像 o1。
- 中间的是图像 o2。
- 右图是图像 o3。



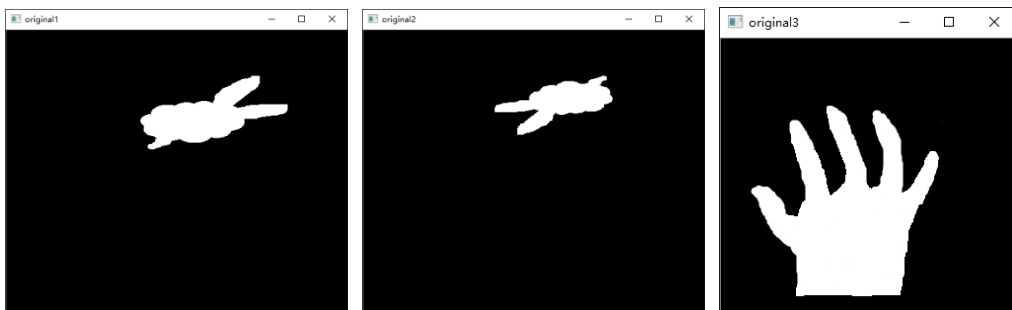


图 12-32 【例 12.27】程序的运行结果

同时，程序还会显示如下的运行结果：

与自身图像的 Hausdorff 距离 $d1 = 0.0$

与旋转缩放后的自身图像的 Hausdorff 距离 $d2 = 18.357559204101562$

与不相似对象的 Hausdorff 距离 $d3 = 57.27128601074219$

从上述运行结果可以看出：

- 相同图像之间的 Hausdorff 距离为零。
- 相似图像之间的 Hausdorff 距离较小。
- 不同图像之间的 Hausdorff 距离较大。

12.7 轮廓的特征值

轮廓自身的一些属性特征及轮廓所包围对象的特征对于描述图像具有重要意义。本节介绍几个轮廓自身的属性特征及轮廓所包围对象的特征。

12.7.1 宽高比

可以使用宽高比（AspectRation）来描述轮廓，例如矩形轮廓的宽高比为：

$$\text{宽高比} = \text{宽度 (Width)} / \text{高度 (Height)}$$

【例 12.28】编写程序计算矩形轮廓的宽高比。

根据题目的要求，编写代码如下：

```
import cv2
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
                                         cv2.RETR_LIST,
                                         cv2.CHAIN_APPROX_SIMPLE)
x,y,w,h = cv2.boundingRect(contours[0])
cv2.rectangle(o,(x,y),(x+w,y+h),(255,255,255),3)
aspectRatio = float(w)/h
```

```
print(aspectRatio)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-33 所示的图像。其中：

- 左图是图像 o。
- 右图显示了图像 o 的矩形轮廓。



图 12-33 【例 12.28】程序的运行结果

同时，程序还会显示如下的运行结果：

```
2.1506849315068495
```

可以看出，轮廓的宽高比约为 2。

12.7.2 Extent

可以使用轮廓面积与矩形边界（矩形包围框、矩形轮廓）面积之比 **Extent** 来描述图像及其轮廓特征。计算方法为：

$$\text{Extent} = \frac{\text{轮廓面积（对象面积）}}{\text{矩形边界面积}}$$

【例 12.29】 计算图像的轮廓面积与其矩形边界面积之比。

根据题目的要求，编写代码如下：

```
import cv2
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)
x,y,w,h = cv2.boundingRect(contours[0])
cv2.drawContours(o, contours[0], -1, (0,0,255), 3)
cv2.rectangle(o, (x,y), (x+w,y+h), (255,0,0), 3)
rectArea=w*h
```

```

cntArea=cv2.contourArea(contours[0])
extend=float(cntArea)/rectArea
print(extend)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-34 所示的图像。其中：

- 左图是图像 o。
- 右图内显示了图像 o 的矩形包围框及轮廓。



图 12-34 【图 12.29】程序的运行结果

同时，程序还会显示如下的运行结果：

```
0.6717127650292296
```

可以看出，本例中图像的轮廓面积与矩形边界面积的比值大约为 0.7。

12.7.3 Solidity

可以使用轮廓面积与凸包面积之比 Solidity 来衡量图像、轮廓及凸包的特征。其计算方法为：

$$\text{Solidity} = \frac{\text{轮廓面积（对象面积）}}{\text{凸包面积}}$$

【例 12.30】编写程序计算图像轮廓面积与凸包面积之比。

根据题目的要求，编写代码如下：

```

import cv2
o = cv2.imread('hand.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy =
cv2.findContours(binary,cv2.RETR_LIST,cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(o,contours[0],-1,(0,0,255),3)
cntArea=cv2.contourArea(contours[0])
hull = cv2.convexHull(contours[0])

```

```

hullArea = cv2.contourArea(hull)
cv2.polylines(o, [hull], True, (0, 255, 0), 2)
solidity=float(cntArea)/hullArea
print(solidity)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-35 所示的图像。其中：

- 左图是图像 o。
- 右图是绘制了凸包和轮廓的图像。

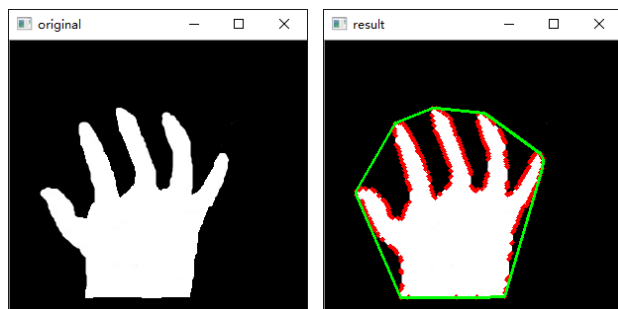


图 12-35 【例 12.30】程序的运行结果

同时，程序还会显示如下的运行结果：

```
0.6752344564084751
```

可以看出，本例中图像的轮廓面积与凸包面积的比值约为 0.7。

12.7.4 等效直径 (Equivalent Diameter)

可以用等效直径来衡量轮廓的特征值，该值是与轮廓面积相等的圆形的直径。其计算公式为：

$$\text{等效直径} = \sqrt{\frac{4 \times \text{轮廓面积}}{\pi}}$$

【例 12.31】 计算与轮廓面积相等的圆形的直径，并绘制与该轮廓等面积的圆。

根据题目的要求，编写代码如下：

```

import cv2
import numpy as np
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

```



```

cv2.drawContours(o, contours[0], -1, (0, 0, 255), 3)
cntArea=cv2.contourArea(contours[0])
equiDiameter = np.sqrt(4*cntArea/np.pi)
print(equiDiameter)
cv2.circle(o, (100,100), int(equiDiameter/2), (0, 0, 255), 3) #展示等直径大小的圆
cv2.imshow("result", o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-36 所示的图像。其中：

- 左图是图像 o。
- 右图显示了图像 o 的轮廓及与该轮廓等面积的圆（显示为灰色）。



图 12-36 【例 12.31】程序的运行结果

同时，程序还会显示如下的运行结果：

```
99.00522529212108
```

可以看出，与本例中与轮廓面积相等的圆形的直径约为 99。

12.7.5 方向

在 OpenCV 中，函数 `cv2.fitEllipse()` 可以用来构造最优拟合椭圆，还可以在返回值内分别返回椭圆的中心点、轴长、旋转角度等信息。使用这种形式，能够更直观地获取椭圆的方向等信息。

函数 `cv2.fitEllipse()` 返回各个属性值的语法格式为：

```
(x,y), (MA,ma), angle = cv2.fitEllipse(cnt)
```

式中几个返回值的意义如下：

- (x,y): 椭圆的中心点。
- (MA,ma): 椭圆水平方向轴和垂直方向轴的长度。
- angle: 椭圆的旋转角度。

【例 12.32】编写程序，观察函数 `cv2.fitEllipse()` 的不同返回值。

根据题目的要求，编写代码如下：

```

import cv2
o = cv2.imread('cc.bmp')

```



```

cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                cv2.RETR_LIST,
                                cv2.CHAIN_APPROX_SIMPLE)

ellipse = cv2.fitEllipse(contours[0])
retval=cv2.fitEllipse(contours[0])
print("单个返回值形式: ")
print("retval=\n",retval)
(x,y),(MA,ma),angle = cv2.fitEllipse(contours[0])
print("三个返回值形式: ")
print("(x,y)=(",x,y,")")
print("(MA,ma)=(",MA,ma,")")
print("angle=",angle)
cv2.ellipse(o,ellipse,(0,0,255),2)
cv2.imshow("result",o)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-37 所示的图像。其中：

- 左图是图像 o。
- 右图是图像 o 的最优拟合椭圆。



图 12-37 【例 12.37】程序的运行结果

同时，程序还会显示如下的运行结果：

单个返回值形式：

retval=

```
((276.2112731933594, 139.6067352294922), (63.01350021362305,
166.72308349609375), 82.60102844238281)
```

三个返回值形式：

```
(x,y)=( 276.2112731933594 139.6067352294922 )
```

```
(MA,ma)=( 63.01350021362305 166.72308349609375 )
```

```
angle= 82.60102844238281
```

从以上运行结果可以看出，函数 cv2.fitEllipse() 以不同形式返回的值是相同的。

12.7.6 掩模和像素点

有时，我们希望获取某对象的掩模图像及其对应的点。12.1.3 节介绍了将函数 `cv2.drawContours()` 的轮廓宽度参数 `thickness` 设置为“-1”，即可获取特定对象的实心轮廓，即特定对象的掩模。

另外，我们可能还希望获取轮廓像素点的具体位置信息。本节介绍如何获取轮廓（实心、空心）的像素点位置信息。

一般情况下，轮廓是图像内非零的像素点，可以通过两种方式获取轮廓像素点的位置信息。一种是使用 Numpy 函数，另外一种是使用 OpenCV 函数。

1. 使用 Numpy 函数获取轮廓像素点

`numpy.nonzero()` 函数能够找出数组内非零元素的位置，但是其返回值是将行、列分别显示的。

例如，对于如下数组 `a` 应用函数 `numpy.nonzero()`：

```
a=
[[0 0 0 1 0]
 [0 0 1 0 1]
 [0 0 1 1 1]
 [1 0 0 0 0]
 [1 0 0 0 1]]
```

返回的数组 `a` 内非零元素的位置信息为：

```
(array([0, 1, 1, 2, 2, 2, 3, 4, 4], dtype=int64), array([3, 2, 4, 2, 3, 4, 0,
0, 4], dtype=int64))
```

使用 `numpy.transpose()` 函数处理上述返回值，则得到这些点的 (x, y) 形式的坐标：

```
[[0 3]
 [1 2]
 [1 4]
 [2 2]
 [2 3]
 [2 4]
 [3 0]
 [4 0]
 [4 4]]
```

【例 12.33】 使用 Numpy 函数获取一个数组内的非零值元素的位置信息。

根据题目要求，编写代码如下：

```
import numpy as np
#-----生成一个元素都是零值的数组 a-----
a=np.zeros((5,5),dtype=np.uint8)
#-----随机将其中 10 个位置上的数值设置为 1-----
#---times 控制次数
#---i,j 是随机生成的行、列位置
```

```

#---a[i,j]=1,将随机挑选出来的位置上的值设置为1
for times in range(10):
    i=np.random.randint(0,5)
    j=np.random.randint(0,5)
    a[i,j]=1
#-----打印数组 a，观察数组 a 内值的情况-----
print("a=\n",a)
#-----查找数组 a 内非零值的位置信息-----
loc=np.transpose(np.nonzero(a))
#-----输出数组 a 内非零值的位置信息-----
print("a 内非零值的位置:\n", loc)

```

运行上述程序，会显示如下的运行结果：

```

a=
[[1 1 0 0 0]
 [1 1 0 1 1]
 [1 0 0 0 0]
 [0 0 0 1 0]
 [1 1 0 0 0]]

```

a 内非零值的位置：

```

[[0 0]
 [0 1]
 [1 0]
 [1 1]
 [1 3]
 [1 4]
 [2 0]
 [3 3]
 [4 0]
 [4 1]]

```

【例 12.34】使用 Numpy 函数获取一个图像内的轮廓点位置。

根据题目的要求，编写代码如下：

```

import cv2
import numpy as np
#-----读取原始图像-----
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
#-----获取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cnt=contours[0]
#-----绘制空心轮廓-----

```



```

mask1 = np.zeros(gray.shape, np.uint8)
cv2.drawContours(mask1, [cnt], 0, 255, 2)
pixelpoints1 = np.transpose(np.nonzero(mask1))
print("pixelpoints1.shape=", pixelpoints1.shape)
print("pixelpoints1=\n", pixelpoints1)
cv2.imshow("mask1", mask1)
#-----绘制实心轮廓-----
mask2 = np.zeros(gray.shape, np.uint8)
cv2.drawContours(mask2, [cnt], 0, 255, -1)
pixelpoints2 = np.transpose(np.nonzero(mask2))
print("pixelpoints2.shape=", pixelpoints2.shape)
print("pixelpoints2=\n", pixelpoints2)
cv2.imshow("mask2", mask2)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-38 所示的图像。其中：

- 左图是图像 o。
- 中间的是空心轮廓图像 mask1。
- 右图是实心轮廓图像 mask2。



图 12-38 【例 12.34】程序的运行结果

同时，程序还会显示如下的运行结果：

```

pixelpoints1.shape= (1400, 2)
pixelpoints1=
[[106 292]
 [106 293]
 [106 294]
 ...
 [180 222]
 [180 223]
 [180 224]]
pixelpoints2.shape= (7892, 2)
pixelpoints2=
[[107 293]
 [107 294]

```

```
[107 295]
...
[179 221]
[179 222]
[179 223]]
```

2. 使用OpenCV函数获取轮廓点

OpenCV 提供了函数 `cv2.findNonZero()` 用于查找非零元素的索引。该函数的语法格式为：

```
idx = cv2.findNonZero( src )
```

式中：

- `idx` 为返回值，表示非零元素的索引位置。需要注意的是，在返回的索引中，每个元素对应的是（列号, 行号）的格式。
- `src` 为参数，表示要查找非零元素的图像。

【例 12.35】 使用 OpenCV 函数 `cv2.findNonZero()` 获取一个数组内的非零值。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
#-----生成一个元素都是零值的数组 a-----
a=np.zeros((5,5),dtype=np.uint8)
#-----随机将其中 10 个位置上的值设置为 1-----
#---times 控制次数
#---i,j 是随机生成的行、列位置
#---a[i,j]=1,将随机挑选出来的位置上的值设置为 1
for times in range(10):
    i=np.random.randint(0,5)
    j=np.random.randint(0,5)
    a[i,j]=1
#-----打印数组 a，观察数组 a 内值的情况-----
print("a=\n",a)
#-----查找数组 a 内非零值的位置信息-----
loc = cv2.findNonZero(a)
#-----输出数组 a 内非零值的位置信息-----
print("a 内非零值的位置:\n", loc)
```

运行上述程序，会显示如下的运行结果：

```
a=
[[0 0 0 0 0]
 [0 0 0 0 1]
 [0 0 1 1 1]
 [1 1 0 0 1]
 [1 1 0 0 0]]
```

a 内非零值的位置：

```
[[[4 1]]
```



```
[[2 2]]
[[3 2]]
[[4 2]]
[[0 3]]
[[1 3]]
[[4 3]]
[[0 4]]
[[1 4]]]
```

【例 12.36】 使用 OpenCV 函数 `cv2.findNonZero()` 获取一个图像内的轮廓点的位置。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
#-----读取原始图像-----
o = cv2.imread('cc.bmp')
cv2.imshow("original",o)
#-----获取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cnt=contours[0]
#-----绘制空心轮廓-----
mask1 = np.zeros(gray.shape,np.uint8)
cv2.drawContours(mask1,[cnt],0,255,2)
pixelpoints1 = cv2.findNonZero(mask1)
print("pixelpoints1.shape=",pixelpoints1.shape)
print("pixelpoints1=\n",pixelpoints1)
cv2.imshow("mask1",mask1)
#-----绘制实心轮廓-----
mask2 = np.zeros(gray.shape,np.uint8)
cv2.drawContours(mask2,[cnt],0,255,-1)
pixelpoints2 = cv2.findNonZero(mask2)
print("pixelpoints2.shape=",pixelpoints2.shape)
print("pixelpoints2=\n",pixelpoints2)
cv2.imshow("mask2",mask2)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-38 所示的图像。其中：

- 左图是图像 o。
- 中间的是空心轮廓图像 mask1。
- 右图是实心轮廓图像 mask2。



图 12-39 【例 12.36】程序的运行结果

同时，程序还会显示如下的运行结果：

```

pixelpoints1.shape= (1400, 1, 2)
pixelpoints1=
[[[292 106]]
 [[293 106]]
 [[294 106]]
 ...

 [[222 180]]
 [[223 180]]
 [[224 180]]]
pixelpoints2.shape= (7892, 1, 2)
pixelpoints2=
[[[293 107]]
 [[294 107]]
 [[295 107]]
 ...

 [[221 179]]
 [[222 179]]
 [[223 179]]]

```

12.7.7 最大值和最小值及它们的位置

OpenCV 提供了函数 `cv2.minMaxLoc()`，用于在指定的对象内查找最大值、最小值及其位置。该函数的语法格式是：

```
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(imgray, mask = mask)
```

式中的返回值为：

- `min_val`: 最小值。
- `max_val`: 最大值。
- `min_loc`: 最小值的位置。
- `max_loc`: 最大值的位置。

式中的参数如下：



- `imggray`: 单通道图像。
- `mask`: 掩模。通过使用掩模图像，可以得到掩模指定区域内的最值信息。

【例 12.37】 使用函数 `cv2.minMaxLoc()` 在图像内查找掩模指定区域内的最大值、最小值及其位置。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
o = cv2.imread('ct.png')
cv2.imshow("original",o)
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy = cv2.findContours(binary,
                                              cv2.RETR_LIST,
                                              cv2.CHAIN_APPROX_SIMPLE)

cnt=contours[2]  #coutours[0]、coutours[1]是左侧字母 R
#-----使用掩模获取感兴趣区域的最值-----
#需要注意函数 minMaxLoc 处理的对象为灰度图像，本例中处理的对象为灰度图像 gray
#如果希望获取彩色图像的最值，需要提取各个通道图像，为每个通道独立计算最值
mask = np.zeros(gray.shape,np.uint8)
mask=cv2.drawContours(mask,[cnt],-1,255,-1)
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(gray,mask = mask)
print("minVal=",minVal)
print("maxVal=",maxVal)
print("minLoc=",minLoc)
print("maxLoc=",maxLoc)
#-----使用掩模获取感兴趣区域并显示-----
masko = np.zeros(o.shape,np.uint8)
masko=cv2.drawContours(masko,[cnt],-1,(255,255,255),-1)
loc=cv2.bitwise_and(o,masko)
cv2.imshow("mask", loc)
#显示灰度结果
#loc=cv2.bitwise_and(gray,mask)
#cv2.imshow("mask", loc)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 12-40 所示的图像。其中：

- 左图是图像 `o`。
- 右图是掩模图像 `mask`。

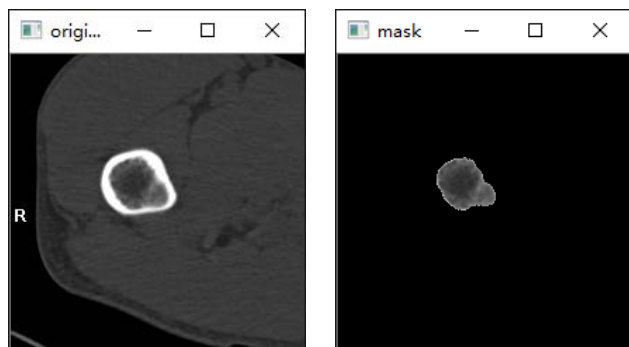


图 12-40 【例 12.37】程序的运行结果

同时，程序还会显示如下的运行结果：

```
minVal= 42.0
maxVal= 200.0
minLoc= (87, 90)
maxLoc= (90, 110)
```

12.7.8 平均颜色及平均灰度

OpenCV 提供了函数 `cv2.mean()`，用于计算一个对象的平均颜色或平均灰度。该函数的语法格式为：

```
mean_val = cv2.mean(im,mask = mask)
```

式中的返回值为 `mean_val`，表示返回的平均值。

式中的参数如下：

- `im`：原图像。
- `mask`：掩模。

【例 12.38】使用函数 `cv2.mean()` 计算一个对象的平均灰度。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
#-----读取并显示原始图像-----
o = cv2.imread('ct.png')
cv2.imshow("original",o)
#-----获取轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image,contours, hierarchy = cv2.findContours(binary,
                                             cv2.RETR_LIST,
                                             cv2.CHAIN_APPROX_SIMPLE)
cnt=contours[2] #coutours[0]、coutours[1]是左侧字母 R
#-----使用掩模获取感兴趣区域的均值-----
mask = np.zeros(gray.shape,np.uint8) #构造 mean 所使用的掩模（必须是单通道的）
```

```

cv2.drawContours(mask, [cnt], 0, (255, 255, 255), -1)
meanVal = cv2.mean(o, mask = mask) # mask 是一个区域，所以必须是单通道的
print("meanVal=\n", meanVal)
#-----使用掩模获取感兴趣区域并显示-----
masko = np.zeros(o.shape, np.uint8)
cv2.drawContours(masko, [cnt], -1, (255, 255, 255), -1)
loc=cv2.bitwise_and(o, masko)
cv2.imshow("mask", loc)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-41 所示的图像。其中：

- 左图是图像 o。
- 右图是获取的感兴趣区域。

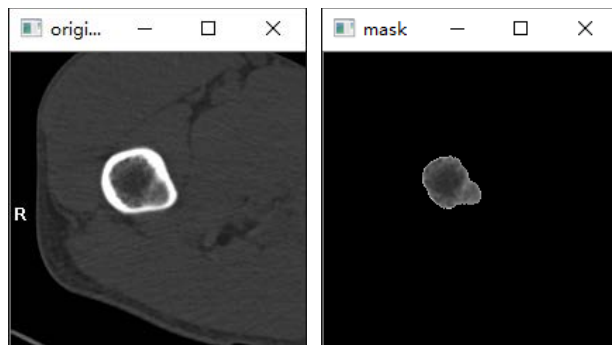


图 12-41 【例 12.38】程序的运行结果

同时，程序还会显示如下的运行结果：

```
meanVal= (85.45594913714805, 85.45594913714805, 85.45594913714805, 0.0)
```

从上述结果可以看出，函数 `cv2.mean()` 能够计算各个通道的均值。上述 4 个值分别是 RGB 和 A 通道（alpha 通道）的均值。本例中，RGB 三个通道的值相同，所以计算出的均值也是一样的。

12.7.9 极点

有时，我们希望获取某个对象内的极值点，例如最左端、最右端、最上端、最下端的四个点。OpenCV 提供了相应的函数来找出这些点，通常的语法格式是：

```

leftmost = tuple(cnt[cnt[:, :, 0].argmin()][0])
rightmost = tuple(cnt[cnt[:, :, 0].argmax()][0])
topmost = tuple(cnt[cnt[:, :, 1].argmin()][0])
bottommost = tuple(cnt[cnt[:, :, 1].argmax()][0])

```

【例 12.39】 计算一幅图像内的极值点。

根据题目的要求，编写代码如下：

```

import cv2
import numpy as np
o = cv2.imread('cs.bmp')
#----- 获取并绘制轮廓-----
gray = cv2.cvtColor(o,cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
image, contours, hierarchy =
    cv2.findContours(binary,cv2.RETR_LIST,cv2.CHAIN_APPROX_SIMPLE)
mask = np.zeros(gray.shape,np.uint8)
cnt=contours[0]
cv2.drawContours(mask, [cnt],0,255,-1)
#----- 计算极值-----
leftmost = tuple(cnt[cnt[:, :, 0].argmin()][0])
rightmost = tuple(cnt[cnt[:, :, 0].argmax()][0])
topmost = tuple(cnt[cnt[:, :, 1].argmin()][0])
bottommost = tuple(cnt[cnt[:, :, 1].argmax()][0])
#----- 打印极值-----
print("leftmost=", leftmost)
print("rightmost=", rightmost)
print("topmost=", topmost)
print("bottommost=", bottommost)
#----- 绘制说明文字-----
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(o, 'A', leftmost, font, 1, (0,0,255),2)
cv2.putText(o, 'B', rightmost, font, 1, (0,0,255),2)
cv2.putText(o, 'C', topmost, font, 1, (0,0,255),2)
cv2.putText(o, 'D', bottommost, font, 1, (0,0,255),2)
#----- 绘制图像-----
cv2.imshow("result",o)
#----- 释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会显示如图 12-42 所示的图像。

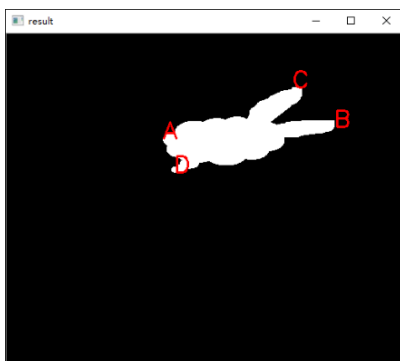


图 12-42 【例 12.39】程序的运行结果

同时，程序还会显示如下的运行结果：

```
leftmost= (202, 135)
rightmost= (423, 120)
topmost= (369, 69)
bottommost= (216, 179)
```